



Department of Electrical Engineering

Inertial Navigation

Assignment 6

Student: Hamed Batani 402101339

Instructor: Prof. Ahi

Q1

Before addressing the specific simulation scenarios, it is crucial to outline the foundational numerical architecture developed to implement random vector generation with a predefined covariance matrix. In stochastic estimation frameworks, mapping mutually independent white Gaussian noise into a designated cross-correlated tensor structure is essential for characterizing process and measurement uncertainties. To achieve accurate results by the book, a multi-variable transformation mapping is executed in MATLAB using a sample size of $N = 10^6$ realizations, ensuring strict empirical convergence to the true analytical limits under the law of large numbers.

The complete simulation code used to execute and verify these methods is provided below:

```
1 %set up
2 N = 1e6;
3 Q = [0.30, 0.15;
4       0.15, 0.55];
5 noiseRaw = randn(2, N);
6 %Q 1.1:
7 Qsqrt = sqrtm(Q);
8 noiseGenerated1 = Qsqrt * noiseRaw;
9 covarianceMatrix1 = cov(noiseGenerated1');
10 %Q 1.2:
11 L = chol(Q, 'lower');
12 noiseGenerated2 = L * noiseRaw;
13 covarianceMatrix2 = cov(noiseGenerated2');
14 %Q 1.3:
15 [V, D] = eig(Q);
16 D_sqrt = sqrt(D);
17 L_eig = V * D_sqrt * V';
18 noiseGenerated3 = L_eig * noiseRaw;
19 covarianceMatrix3 = cov(noiseGenerated3');
20 %Q1.4:
```

```

21 [V, D] = eig(Q);
22 D_sqrt = sqrt(D);
23 T = V * D_sqrt;
24 noiseGenerated4 = T * noiseRaw;
25 covarianceMatrix4 = cov(noiseGenerated4');
26 %display
27 disp('Target covariance matrix Q:');
28 disp(Q);
29 disp('1.1: Method 1 (sqrtm) - Resulting covariance matrix:');
30 disp(covarianceMatrix1);
31 disp('1.2: Method 2 (chol, lower) - Resulting covariance matrix:');
32 disp(covarianceMatrix2);
33 disp('1.3: Method 3 (eig decomposition) - Resulting covariance matrix:');
34 disp(covarianceMatrix3);
35 disp('1.4: Method 4 (Simon''s algorithm) - Resulting covariance matrix:');
36 disp(covarianceMatrix4);
37 % 1.5:
38 R = [0.10, 0.05;
39       0.05, 0.20];
40 M = [0.05, 0.04;
41       0.04, 0.10];
42 JointCov = [Q, M;
43             M', R];
44 [V_joint, D_joint] = eig(JointCov);
45 D_joint_sqrt = sqrt(D_joint);
46 T_joint = V_joint * D_joint_sqrt;
47 jointNoiseRaw = randn(4, N);
48 jointNoiseGenerated = T_joint * jointNoiseRaw;
49 processNoise = jointNoiseGenerated(1:2, :);
50 measurementNoise = jointNoiseGenerated(3:4, :);
51 covarianceMatrix_joint_check = cov(jointNoiseGenerated');
52 covarianceMatrix_Q_check = covarianceMatrix_joint_check(1:2, 1:2);
53 covarianceMatrix_R_check = covarianceMatrix_joint_check(3:4, 3:4);
54 covarianceMatrix_M_check = covarianceMatrix_joint_check(1:2, 3:4);
55 disp('1.5: Process noise covariance check (should match Q):');
56 disp(covarianceMatrix_Q_check);
57 disp('1.5: Measurement noise covariance check (should match R):');
58 disp(covarianceMatrix_R_check);
59 disp('1.5: Cross-covariance check (should match M):');
60 disp(covarianceMatrix_M_check);
61 %now we rank them
62 errors = [norm(covarianceMatrix1 - Q, 'fro'), norm(covarianceMatrix2 - Q, 'fro'
        ), norm(covarianceMatrix3 - Q, 'fro'), norm(covarianceMatrix4 - Hardcoded, '
        fro')];
63 [~, rankOrder] = sort(errors);
64 methodNames = {'1.1 (sqrtm)', '1.2 (chol)', '1.3 (eig)', '1.4 (Simon)'};
65 disp('Ranking from closest to farthest from Q:');
66 for i = 1:length(rankOrder)
67     fprintf('%d. %s - error: %.6e\n', i, methodNames{rankOrder(i)}, errors(
        rankOrder(i)));
68 end

```

Listing 1: Complete MATLAB Script for Homework 6 Problem 1 Verification

1.1

In this section, the code utilizes the principal matrix square root function via `Qsqrt = sqrtm(Q)` to determine the transformation matrix T . This works because it directly computes a unique positive semi-definite matrix satisfying $TT = Q$. Left-multiplying this matrix by our raw white noise vector array scales the variables perfectly into the target correlation boundaries.

The resulting empirical matrix from the command window is:

```
1.1: Method 1 (sqrtm) - Resulting covariance matrix:
    0.2998    0.1491
    0.1491    0.5499
```

QUESTION: What is the difference between the `rand` and `randn` commands in MATLAB?

ANSWER: The `rand` command draws pseudo-random samples from a uniform distribution bounded strictly over the open interval $(0, 1)$. Conversely, the `randn` function generates samples from a standard normal distribution possessing an analytical mean of $\mu = 0$ and a variance of $\sigma^2 = 1$. Because standard state estimation filters assume a white Gaussian noise profile, `randn` is the mathematically required primitive choice here.

1.2

In this section, the code computes a lower triangular matrix L via Cholesky factorization using the syntax `L = chol(Q, 'lower')`. This decomposes the matrix such that $Q = LL^T$. The raw noise matrix is transformed via left-multiplication as `L * noiseRaw`, which provides an efficient, lower-triangular structural solution.

The resulting empirical matrix from the command window is:

```
1.2: Method 2 (chol, lower) - Resulting covariance matrix:
    0.3000    0.1492
    0.1492    0.5494
```

QUESTION: If you used the upper setting instead of lower in the Cholesky decomposition, what difference would it make?

ANSWER: Specifying `chol(Q, 'upper')` calculates a unique upper triangular matrix U where the decomposition is mapped as $Q = U^T U$. Because our transformation layout requires the form $Q = TT^T$, switching to the upper configuration yields $T = U^T$. To maintain identical covariance propagation properties, the multiplication syntax in the code would need to be modified to use a transpose: `noiseGenerated2 = U' * noiseRaw`.

1.3

In this section, the code extracts the orthonormal eigenvectors and diagonal eigenvalue matrices using `[V, D] = eig(Q)`. It computes the element-wise square root of the eigenvalues and forms a symmetric

square root matrix transformation using the expression $L_{\text{eig}} = V * D_{\text{sqrt}} * V'$. This transforms the raw noise vector accurately into the coordinate frames of the principal axes.

The resulting empirical matrix from the command window is:

```
1.3: Method 3 (eig decomposition) - Resulting covariance matrix:
    0.2998    0.1491
    0.1491    0.5499
```

QUESTION: What is the difference between calculating the matrix square root using $D_{\text{sqrt}} = \text{sqrt}(D);$, $D_{\text{sqrt}} = \text{sqrtn}(D);$, and $D_{\text{sqrt}} = D^{0.5};$?

ANSWER: The primitive $\text{sqrt}(D)$ applies a scalar element-wise square root across each standalone entry in the array. Because D is structurally a diagonal matrix of eigenvalues, an element-wise operation is mathematically correct and computationally optimal. Alternatively, $\text{sqrtn}(D)$ explicitly targets the principal matrix square root using generalized algorithmic matrix-level calculations. Finally, the matrix power expression $D^{0.5}$ evaluates the fractional power of a matrix, which in MATLAB maps to the exact same numerical routine as $\text{sqrtn}(D)$.

1.4

In this section, the code implements the directional decomposition mapping sequence proposed in Simon's textbook. Using the components from the spectral step, the mapping operator is set directly as $T = V * D_{\text{sqrt}}$. This omits the final eigenvector post-multiplication layer while remaining fully valid since $TT^T = VD^{1/2}D^{1/2}V^T = Q$.

The resulting empirical matrix from the command window is:

```
1.4: Method 4 (Simon's algorithm) - Resulting covariance matrix:
    0.3007    0.1507
    0.1507    0.5496
```

QUESTION: Can the `jordan` command be used to solve this problem?

ANSWER: Theoretically, yes, because a valid covariance matrix Q is real and symmetric, making it fully diagonalizable. Its Jordan canonical form collapses back into a standard diagonal matrix of eigenvalues, which is mathematically identical to the `eig` routine. Practically, however, the answer is no; MATLAB's `jordan` command uses symbolic variables to find exact roots. This process is highly sensitive to floating-point perturbations, can introduce numerical instability, and adds immense computational latency.

1.5

In this final section, the code scales the routine to handle jointly correlated process and measurement matrices where $M \neq 0$. It creates an augmented block matrix array via $\text{JointCov} = [Q, M; M', R]$, computes its overall spectral factorization components, and generates a $4 \times N$ joint noise array. Row index slicing separates the vectors into distinct process and measurement noise distributions while preserving their precise cross-coupling characteristics.

The empirical verification checkpoints from the command window yield:

1.5: Process noise covariance check (should match Q):

0.3000	0.1499
0.1499	0.5494

1.5: Measurement noise covariance check (should match R):

0.1000	0.0500
0.0500	0.1999

1.5: Cross-covariance check (should match M):

0.0499	0.0395
0.0404	0.1000

To systematically evaluate the performance of each independent layout, the script computes the matrix Frobenius norm error profile. The sorting sequence from closest to farthest from Q evaluates as:

1. **1.4 (Simon's Method):** Error profile equals 1.257594×10^{-3}
2. **1.1 (Matrix square root function sqrtm):** Error profile equals 1.262420×10^{-3}
3. **1.3 (Spectral decomposition eig):** Error profile equals 1.262420×10^{-3}
4. **1.2 (Triangular Cholesky factorization):** Error profile equals 1.334570×10^{-3}

Q2

Overview

The problem under consideration is the recursive Bayesian state estimation of a planar unicycle-type mobile robot, observed through a pair of heterogeneous, multi-rate, nonlinear sensors (a range sensor and a bearing sensor). Since both the process model and the measurement model are nonlinear in the state vector, the linear Kalman filter formalism is not directly applicable, and the Extended Kalman Filter (EKF) is instead employed as a first-order (Taylor-linearized) approximation to the optimal nonlinear filter. The implementation strictly follows the recursive prediction/update structure summarized in **pages 63–64** of the course slides, namely

$$\begin{aligned}\hat{\mathbf{x}}_k^- &= f_{k-1}(\hat{\mathbf{x}}_{k-1}^+, \mathbf{u}_{k-1}, \mathbf{0}) \textit{Time} \\ P_k^- &= F_{k-1} P_{k-1}^+ F_{k-1}^T + L_{k-1} Q_{k-1} L_{k-1}^T \\ K_k &= P_k^- H_k^T [H_k P_k^- H_k^T + M_k R_k M_k^T]^{-1} \\ \hat{\mathbf{x}}_k^+ &= \hat{\mathbf{x}}_k^- + K_k [\mathbf{y}_k - h_k(\hat{\mathbf{x}}_k^-, \mathbf{0})] \textit{Measurement} \\ P_k^+ &= (I - K_k H_k) P_k^-\end{aligned}$$

with the Jacobians $F_{k-1} = \partial f / \partial \mathbf{x} \big|_{\hat{\mathbf{x}}_{k-1}^+}$ and $H_k = \partial h / \partial \mathbf{x} \big|_{\hat{\mathbf{x}}_k^-}$ evaluated analytically at every time step, since both are obtainable in closed form for this plant and measurement model, avoiding the additional approximation error that a numerical Jacobian would introduce. Throughout this section, $\Delta t = 0.05$ s is used as the base integration step, and all multi-rate sensor schedules are defined as integer multiples of this base step, consistent with the discretization philosophy of **page 58** and the multi-rate sensor-fusion discussion of **slide 120**.

The complete simulation code used to execute and verify all seven sub-parts is provided below.

```
1
2 %2.1
3 f = @(x, u) [u(1)*cos(x(3));
4             u(1)*sin(x(3));
5             u(2)];
6
7 % as said in class , this could be seen as our tuning parametr as well
8 Qc = diag([0.05^2, 0.05^2, 0.03^2]);
9
10 %2.2
11 v = @(t) cos(t);
12 omega = @(t) sin(0.2*t);
13 x0 = [0; 1; pi/2];
14
15 %2.3
16 dt = 0.05;
17 T_total = 240;
18 N = T_total / dt;
19 t = (0:N) * dt;
20 x_true = zeros(3, N+1);
21 x_true(:,1) = x0;
22
23 Qd = Qc * dt;
24 Qd_sqrt = sqrtm(Qd);
25 rng(1);
26 w = Qd_sqrt * randn(3, N);
27
28 for k = 1:N
29     u_k = [v(t(k)); omega(t(k))];
30     x_true(:,k+1) = x_true(:,k) + dt * f(x_true(:,k), u_k) + w(:,k);
31 end
32
33 %2.4
34 h = @(x) [sqrt(x(1)^2 + x(2)^2); % range
35          atan2(x(2), x(1))]; % bearing
36
37
38 Rd_range = 0.1^2;
39 Rd_bearing = 0.1^2;
40
41 sigma_x = 0.1;
42 sigma_y = 0.1;
43 sigma_theta = 0.05;
44 P0 = diag([sigma_x^2, sigma_y^2, sigma_theta^2]);
45
46 x_hat0 = [1; 0; deg2rad(120)];
47 x_hat_plus = zeros(3, N+1);
48 P_plus = zeros(3, 3, N+1);
49 x_hat_plus(:,1) = x_hat0;
50 P_plus(:,:,1) = P0;
51
52 y_meas = NaN(2, N+1);
53
54 %this is the multi rate sensors we talked about
55 steps_1s = round(1.0 / dt); % bearing every 1 s
```

```

56 steps_2s = round(2.0 / dt); % range every 2 s
57
58
59 rng(2);
60 v_range_all = sqrt(Rd_range) * randn(1, N+1);
61 v_bearing_all = sqrt(Rd_bearing) * randn(1, N+1);
62
63 innovations = NaN(2, N+1);
64 S_matrices = zeros(2, 2, N+1);
65
66 for k = 1:N
67     u_k = [v(t(k)); omega(t(k))];
68     theta_prev = x_hat_plus(3, k);
69
70     F_km1 = eye(3) + dt * [0, 0, -u_k(1)*sin(theta_prev);
71                           0, 0, u_k(1)*cos(theta_prev);
72                           0, 0, 0];
73     L_km1 = eye(3);
74
75     x_hat_minus = x_hat_plus(:,k) + dt * f(x_hat_plus(:,k), u_k);
76     P_minus = F_km1 * P_plus(:, :, k) * F_km1' + L_km1 * Qd * L_km1';
77
78     is_bearing_step = (mod(k, steps_1s) == 0);
79     is_range_step = (mod(k, steps_2s) == 0);
80
81     if is_range_step && is_bearing_step
82         % both range and bearing updates available
83         x_true_k = x_true(:, k+1);
84         v_k = [v_range_all(k+1); v_bearing_all(k+1)];
85         y_k = h(x_true_k) + v_k;
86         y_meas(:,k+1) = y_k;
87
88         rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
89         H_k = [ x_hat_minus(1)/rho, x_hat_minus(2)/rho, 0;
90               -x_hat_minus(2)/rho^2, x_hat_minus(1)/rho^2, 0];
91
92         S_k = H_k * P_minus * H_k' + diag([Rd_range, Rd_bearing]);
93         K_k = P_minus * H_k' / S_k;
94
95         nu = y_k - h(x_hat_minus);
96         nu(2) = atan2(sin(nu(2)), cos(nu(2))); % safe angular wrapping
97
98         innovations(:,k+1) = nu;
99         S_matrices(:, :, k+1) = S_k;
100        x_hat_plus(:,k+1) = x_hat_minus + K_k * nu;
101        P_plus(:, :, k+1) = (eye(3) - K_k * H_k) * P_minus;
102
103    elseif is_bearing_step
104        % only bearing update available (every 1 s)
105        x_true_k = x_true(:, k+1);
106        v_k = v_bearing_all(k+1);
107        y_k = atan2(x_true_k(2), x_true_k(1)) + v_k;
108        y_meas(2,k+1) = y_k;
109
110        rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
111        H_k = [-x_hat_minus(2)/rho^2, x_hat_minus(1)/rho^2, 0];

```

```

112     S_k = H_k * P_minus * H_k' + Rd_bearing;
113     K_k = P_minus * H_k' / S_k;
114
115
116     nu = y_k - atan2(x_hat_minus(2), x_hat_minus(1));
117     nu = atan2(sin(nu), cos(nu)); % safe angular wrapping
118
119     innovations(2,k+1) = nu;
120     S_matrices(2,2,k+1) = S_k;
121     x_hat_plus(:,k+1) = x_hat_minus + K_k * nu;
122     P_plus(:,:,k+1) = (eye(3) - K_k * H_k) * P_minus;
123
124     else
125
126     x_hat_plus(:,k+1) = x_hat_minus;
127     P_plus(:,:,k+1) = P_minus;
128     end
129 end
130
131
132 r_true = sqrt(x_true(1,:).^2 + x_true(2,:).^2);
133 phi_true = atan2(x_true(2,:), x_true(1,:));
134 t_zoom = 30;
135 idx_zoom = t <= t_zoom;
136
137
138 save_dir_24 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
139 InertialNavigation\HWs\HW6\SimulationFigures\2\2.4';
140 if ~exist(save_dir_24, 'dir'), mkdir(save_dir_24); end
141
142 y_state_meas = NaN(3, N+1);
143
144 both_idx = find(~isnan(y_meas(1,:)) & ~isnan(y_meas(2,:)));
145 y_state_meas(1, both_idx) = y_meas(1, both_idx) .* cos(y_meas(2, both_idx));
146 y_state_meas(2, both_idx) = y_meas(1, both_idx) .* sin(y_meas(2, both_idx));
147
148 fig1 = figure;
149 subplot(3,1,1);
150 plot(t(idx_zoom), x_true(1,idx_zoom), 'b'); hold on;
151 plot(t(idx_zoom), x_hat_plus(1,idx_zoom), 'r--');
152 plot(t(idx_zoom), y_state_meas(1,idx_zoom), 'g.', 'MarkerSize', 10);
153 xlabel('Time [s]'); ylabel('x [m]');
154 legend('Ground Truth', 'A Posteriori Estimate', 'Measurement'); grid on;
155 title('Part 2.4: State x - Zoomed View');
156 subplot(3,1,2);
157 plot(t(idx_zoom), x_true(2,idx_zoom), 'b'); hold on;
158 plot(t(idx_zoom), x_hat_plus(2,idx_zoom), 'r--');
159 plot(t(idx_zoom), y_state_meas(2,idx_zoom), 'g.', 'MarkerSize', 10);
160 xlabel('Time [s]'); ylabel('y [m]');
161 legend('Ground Truth', 'A Posteriori Estimate', 'Measurement'); grid on;
162 title('Part 2.4: State y - Zoomed View');
163 subplot(3,1,3);
164 plot(t(idx_zoom), x_true(3,idx_zoom), 'b'); hold on;
165 plot(t(idx_zoom), x_hat_plus(3,idx_zoom), 'r--');
166 xlabel('Time [s]'); ylabel('\theta [rad]');

```

```

167 legend('Ground Truth', 'A Posteriori Estimate'); grid on;
168 title('Part 2.4: Heading \theta - Zoomed View (no direct \theta measurement)');
169 saveas(fig1, fullfile(save_dir_24, 'states_combined_zoomed.png'));
170
171 % --- Figure 2: Part 2.4 - Full combined view (truth + a posteriori +
    measurement) ---
172 fig2 = figure;
173 subplot(3,1,1);
174 plot(t, x_true(1,:), 'b'); hold on;
175 plot(t, x_hat_plus(1,:), 'r--');
176 plot(t, y_state_meas(1,:), 'g.', 'MarkerSize', 6);
177 xlabel('Time [s]'); ylabel('x [m]');
178 legend('Ground Truth', 'A Posteriori Estimate', 'Measurement'); grid on;
179 title('Part 2.4: State x - Full History');
180 subplot(3,1,2);
181 plot(t, x_true(2,:), 'b'); hold on;
182 plot(t, x_hat_plus(2,:), 'r--');
183 plot(t, y_state_meas(2,:), 'g.', 'MarkerSize', 6);
184 xlabel('Time [s]'); ylabel('y [m]');
185 legend('Ground Truth', 'A Posteriori Estimate', 'Measurement'); grid on;
186 title('Part 2.4: State y - Full History');
187 subplot(3,1,3);
188 plot(t, x_true(3,:), 'b'); hold on;
189 plot(t, x_hat_plus(3,:), 'r--');
190 xlabel('Time [s]'); ylabel('\theta [rad]');
191 legend('Ground Truth', 'A Posteriori Estimate'); grid on;
192 title('Part 2.4: Heading \theta - Full History (no direct \theta measurement)');
    ;
193 saveas(fig2, fullfile(save_dir_24, 'states_combined_full.png'));
194
195
196 fig3 = figure;
197 subplot(2,1,1);
198 plot(t, r_true, 'b'); hold on;
199 plot(t, y_meas(1,:), 'g.', 'MarkerSize', 8);
200 xlabel('Time [s]'); ylabel('r [m]');
201 legend('Ground Truth', 'Measurement'); grid on;
202 title('Part 2.4: Range Measurements (r)');
203 subplot(2,1,2);
204 plot(t, phi_true, 'b'); hold on;
205 plot(t, y_meas(2,:), 'g.', 'MarkerSize', 8);
206 xlabel('Time [s]'); ylabel('\phi [rad]');
207 legend('Ground Truth', 'Measurement'); grid on;
208 title('Part 2.4: Bearing Measurements (\phi)');
209 saveas(fig3, fullfile(save_dir_24, 'measurements_polar.png'));
210
211 %2.5
212 save_dir_25 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    InertialNavigation\HWs\HW6\SimulationFigures\2\2.5';
213 if ~exist(save_dir_25, 'dir'), mkdir(save_dir_25); end
214 errors_state = x_true - x_hat_plus;
215 errors_state(3,:) = atan2(sin(errors_state(3,:)), cos(errors_state(3,:)));
216
217 sigma_bounds = zeros(3, N+1);
218 for k = 1:(N+1)
219     sigma_bounds(1,k) = sqrt(P_plus(1,1,k));

```

```

220     sigma_bounds(2,k) = sqrt(P_plus(2,2,k));
221     sigma_bounds(3,k) = sqrt(P_plus(3,3,k));
222 end
223
224 fig4 = figure;
225 subplot(3,1,1);
226 plot(t, errors_state(1,:), 'b'); hold on;
227 plot(t, 3*sigma_bounds(1,:), 'r--', t, -3*sigma_bounds(1,:), 'r--');
228 xlabel('Time [s]'); ylabel('x Error [m]');
229 legend('Estimation Error', '\pm3\sigma Bound'); grid on;
230 title('Part 2.5: Estimation Error for Position x with Envelopes');
231 subplot(3,1,2);
232 plot(t, errors_state(2,:), 'b'); hold on;
233 plot(t, 3*sigma_bounds(2,:), 'r--', t, -3*sigma_bounds(2,:), 'r--');
234 xlabel('Time [s]'); ylabel('y Error [m]');
235 legend('Estimation Error', '\pm3\sigma Bound'); grid on;
236 title('Part 2.5: Estimation Error for Position y with Envelopes');
237 subplot(3,1,3);
238 plot(t, errors_state(3,:), 'b'); hold on;
239 plot(t, 3*sigma_bounds(3,:), 'r--', t, -3*sigma_bounds(3,:), 'r--');
240 xlabel('Time [s]'); ylabel('\theta Error [rad]');
241 legend('Estimation Error', '\pm3\sigma Bound'); grid on;
242 title('Part 2.5: Estimation Error for Heading \theta with Envelopes');
243 saveas(fig4, fullfile(save_dir_25, 'error_bounds.png'));
244
245 %2.6
246 save_dir_26 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    InertialNavigation\HWs\HW6\SimulationFigures\2\2.6';
247 if ~exist(save_dir_26, 'dir'), mkdir(save_dir_26); end
248
249 t_outage_start = 60;
250 t_outage_dur   = 20;
251 t_outage_end   = t_outage_start + t_outage_dur;
252 k_outage_start = round(t_outage_start/dt) + 1;
253 k_outage_end   = round(t_outage_end/dt) + 1;
254
255 state_names = {'x [m]', 'y [m]', '\theta [rad]'};
256
257
258 scenarios = {'range_only', [true, false]; ...
259             'bearing_only', [false, true]; ...
260             'both', [true, true]};
261
262 for s_idx = 1:size(scenarios,1)
263     scen_name = scenarios{s_idx,1};
264     block_range = scenarios{s_idx,2}(1);
265     block_bearing = scenarios{s_idx,2}(2);
266
267     x_hat_26 = zeros(3, N+1);
268     P_26     = zeros(3, 3, N+1);
269     x_hat_26(:,1) = x_hat0;
270     P_26(:,:,1) = P0;
271
272     for k = 1:N
273         u_k = [v(t(k)); omega(t(k))];
274         theta_prev = x_hat_26(3, k);

```

```

275     F_km1 = eye(3) + dt * [0, 0, -u_k(1)*sin(theta_prev);
276                           0, 0, u_k(1)*cos(theta_prev);
277                           0, 0, 0];
278     x_hat_minus = x_hat_26(:,k) + dt * f(x_hat_26(:,k), u_k);
279     P_minus = F_km1 * P_26(:, :, k) * F_km1' + Qd;
280
281     in_outage = (k+1 >= k_outage_start) && (k+1 <= k_outage_end);
282
283     is_bearing_step = (mod(k, steps_1s) == 0) && ~(in_outage &&
block_bearing);
284     is_range_step = (mod(k, steps_2s) == 0) && ~(in_outage && block_range
);
285
286     if is_range_step && is_bearing_step
287         x_true_k = x_true(:, k+1);
288         v_k = [v_range_all(k+1); v_bearing_all(k+1)];
289         y_k = h(x_true_k) + v_k;
290         rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
291         H_k = [ x_hat_minus(1)/rho, x_hat_minus(2)/rho, 0;
292               -x_hat_minus(2)/rho^2, x_hat_minus(1)/rho^2, 0];
293         S_k = H_k * P_minus * H_k' + diag([Rd_range, Rd_bearing]);
294         K_k = P_minus * H_k' / S_k;
295         nu = y_k - h(x_hat_minus);
296         nu(2) = atan2(sin(nu(2)), cos(nu(2)));
297         x_hat_26(:,k+1) = x_hat_minus + K_k * nu;
298         P_26(:, :, k+1) = (eye(3) - K_k * H_k) * P_minus;
299
300     elseif is_bearing_step
301         x_true_k = x_true(:, k+1);
302         v_k = v_bearing_all(k+1);
303         y_k = atan2(x_true_k(2), x_true_k(1)) + v_k;
304         rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
305         H_k = [-x_hat_minus(2)/rho^2, x_hat_minus(1)/rho^2, 0];
306         S_k = H_k * P_minus * H_k' + Rd_bearing;
307         K_k = P_minus * H_k' / S_k;
308         nu = y_k - atan2(x_hat_minus(2), x_hat_minus(1));
309         nu = atan2(sin(nu), cos(nu));
310         x_hat_26(:,k+1) = x_hat_minus + K_k * nu;
311         P_26(:, :, k+1) = (eye(3) - K_k * H_k) * P_minus;
312
313     elseif is_range_step
314         x_true_k = x_true(:, k+1);
315         v_k = v_range_all(k+1);
316         y_k = sqrt(x_true_k(1)^2 + x_true_k(2)^2) + v_k;
317         rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
318         H_k = [x_hat_minus(1)/rho, x_hat_minus(2)/rho, 0];
319         S_k = H_k * P_minus * H_k' + Rd_range;
320         K_k = P_minus * H_k' / S_k;
321         nu = y_k - rho;
322         x_hat_26(:,k+1) = x_hat_minus + K_k * nu;
323         P_26(:, :, k+1) = (eye(3) - K_k * H_k) * P_minus;
324
325     else
326
327         x_hat_26(:,k+1) = x_hat_minus;
328         P_26(:, :, k+1) = P_minus;

```

```

329     end
330 end
331
332 fig_outage = figure;
333 for i = 1:3
334     subplot(3,1,i);
335     plot(t, x_true(i,:), 'b'); hold on;
336     plot(t, x_hat_26(i,:), 'r--');
337     yl = ylim;
338     patch([t_outage_start t_outage_end t_outage_end t_outage_start], ...
339           [yl(1) yl(1) yl(2) yl(2)], [1 0.85 0.85], 'EdgeColor', 'none', '
FaceAlpha', 0.5);
340     uistack(findobj(gca,'Type','patch'), 'bottom');
341     xlabel('Time [s]'); ylabel(state_names{i});
342     legend('Ground Truth', 'A Posteriori Estimate', 'No-Measurement Window'
); grid on;
343     title(['Part 2.6 (' strrep(scen_name, '_', ' ') '): State ' state_names{i
}]);
344 end
345 saveas(fig_outage, fullfile(save_dir_26, ['outage_' scen_name '.png']));
346 end
347
348
349
350 %2.7
351 save_dir_27 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
InertialNavigation\HWs\HW6\SimulationFigures\2\2.7';
352 if ~exist(save_dir_27, 'dir'), mkdir(save_dir_27); end
353
354 T_delay      = 4.0;
355 steps_delay  = round(T_delay / dt);
356
357
358 x_ideal = x_hat_plus;
359
360
361 x_naive = zeros(3, N+1);
362 P_naive = zeros(3, 3, N+1);
363 x_naive(:,1) = x_hat0;
364 P_naive(:,:,1) = P0;
365
366 for k = 1:N
367     u_k      = [v(t(k)); omega(t(k))];
368     theta_prev = x_naive(3, k);
369     F_km1 = eye(3) + dt * [0, 0, -u_k(1)*sin(theta_prev);
370                           0, 0,  u_k(1)*cos(theta_prev);
371                           0, 0,  0
];
372     x_hat_minus = x_naive(:,k) + dt * f(x_naive(:,k), u_k);
373     P_minus     = F_km1 * P_naive(:,:,k) * F_km1' + Qd;
374
375     is_bearing_step = (mod(k, steps_1s) == 0);
376     is_range_step   = (mod(k, steps_2s) == 0);
377
378     range_val = NaN;
379     if is_range_step
380         k_delayed = max(k+1 - steps_delay, 1); % time the range value actually

```

```

describes
381     range_val = sqrt(x_true(1,k_delayed)^2 + x_true(2,k_delayed)^2) +
v_range_all(k+1);
382 end
383
384 if is_range_step && is_bearing_step
385     bearing_val = atan2(x_true(2,k+1), x_true(1,k+1)) + v_bearing_all(k+1);
386     y_k = [range_val; bearing_val];
387     rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
388     H_k = [ x_hat_minus(1)/rho,    x_hat_minus(2)/rho,    0;
389           -x_hat_minus(2)/rho^2, x_hat_minus(1)/rho^2, 0];
390     S_k = H_k * P_minus * H_k' + diag([Rd_range, Rd_bearing]);
391     K_k = P_minus * H_k' / S_k;
392     nu = y_k - h(x_hat_minus);
393     nu(2) = atan2(sin(nu(2)), cos(nu(2)));
394     x_naive(:,k+1) = x_hat_minus + K_k * nu;
395     P_naive(:,:,k+1) = (eye(3) - K_k * H_k) * P_minus;
396
397 elseif is_bearing_step
398     bearing_val = atan2(x_true(2,k+1), x_true(1,k+1)) + v_bearing_all(k+1);
399     rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
400     H_k = [-x_hat_minus(2)/rho^2, x_hat_minus(1)/rho^2, 0];
401     S_k = H_k * P_minus * H_k' + Rd_bearing;
402     K_k = P_minus * H_k' / S_k;
403     nu = bearing_val - atan2(x_hat_minus(2), x_hat_minus(1));
404     nu = atan2(sin(nu), cos(nu));
405     x_naive(:,k+1) = x_hat_minus + K_k * nu;
406     P_naive(:,:,k+1) = (eye(3) - K_k * H_k) * P_minus;
407
408 elseif is_range_step
409     % range-only instant, naively applied as if current
410     rho = sqrt(x_hat_minus(1)^2 + x_hat_minus(2)^2);
411     H_k = [x_hat_minus(1)/rho, x_hat_minus(2)/rho, 0];
412     S_k = H_k * P_minus * H_k' + Rd_range;
413     K_k = P_minus * H_k' / S_k;
414     nu = range_val - rho;
415     x_naive(:,k+1) = x_hat_minus + K_k * nu;
416     P_naive(:,:,k+1) = (eye(3) - K_k * H_k) * P_minus;
417
418 else
419     x_naive(:,k+1) = x_hat_minus;
420     P_naive(:,:,k+1) = P_minus;
421 end
422 end
423
424
425 x_comp = zeros(3, N+1);
426 P_comp = zeros(3, 3, N+1);
427 x_comp(:,1) = x_hat0;
428 P_comp(:,:,1) = P0;
429
430 for k = 1:N
431     u_k = [v(t(k)); omega(t(k))];
432     theta_prev = x_comp(3, k);
433     F_km1 = eye(3) + dt * [0,    0,   -u_k(1)*sin(theta_prev);
434                          0,    0,    u_k(1)*cos(theta_prev);

```

```

435         0, 0, 0];
436 x_hat_minus = x_comp(:,k) + dt * f(x_comp(:,k), u_k);
437 P_minus     = F_km1 * P_comp(:,:,k) * F_km1' + Qd;
438
439 is_bearing_step = (mod(k, steps_1s) == 0);
440 is_range_step   = (mod(k, steps_2s) == 0);
441
442 bearing_val = NaN;
443 if is_bearing_step
444     bearing_val = atan2(x_true(2,k+1), x_true(1,k+1)) + v_bearing_all(k+1);
445 end
446
447 if is_range_step
448     % the range value reports the state from steps_delay samples ago
449     k_delayed = max(k+1 - steps_delay, 1);
450     range_val = sqrt(x_true(1,k_delayed)^2 + x_true(2,k_delayed)^2) +
v_range_all(k+1);
451
452
453 x_retro = x_hat_minus;
454 for s = 1:steps_delay
455     t_back = t(k+1) - (s-1)*dt;
456     u_back = [v(t_back); omega(t_back)];
457     x_retro = x_retro - dt * f(x_retro, u_back);
458 end
459
460
461 rho_r = sqrt(x_retro(1)^2 + x_retro(2)^2);
462 H_r   = [x_retro(1)/rho_r, x_retro(2)/rho_r, 0];
463 P_r   = P_minus;
464 S_r   = H_r * P_r * H_r' + Rd_range;
465 K_r   = P_r * H_r' / S_r;
466 nu_r  = range_val - rho_r;
467 x_retro_updated = x_retro + K_r * nu_r;
468
469
470 x_repred = x_retro_updated;
471 for s = 1:steps_delay
472     t_fwd = t(k+1) - steps_delay*dt + (s-1)*dt;
473     u_fwd = [v(t_fwd); omega(t_fwd)];
474     x_repred = x_repred + dt * f(x_repred, u_fwd);
475 end
476
477 x_after_range = x_repred;
478
479 P_after_range = (eye(3) - K_r * H_r) * P_minus;
480 else
481     x_after_range = x_hat_minus;
482     P_after_range = P_minus;
483 end
484
485 if is_bearing_step
486
487     rho_b = sqrt(x_after_range(1)^2 + x_after_range(2)^2);
488     H_b   = [-x_after_range(2)/rho_b^2, x_after_range(1)/rho_b^2, 0];
489     S_b   = H_b * P_after_range * H_b' + Rd_bearing;

```

```

490     K_b    = P_after_range * H_b' / S_b;
491     nu_b   = bearing_val - atan2(x_after_range(2), x_after_range(1));
492     nu_b   = atan2(sin(nu_b), cos(nu_b));
493     x_comp(:,k+1) = x_after_range + K_b * nu_b;
494     P_comp(:,:,k+1) = (eye(3) - K_b * H_b) * P_after_range;
495     else
496         x_comp(:,k+1) = x_after_range;
497         P_comp(:,:,k+1) = P_after_range;
498     end
499 end
500
501 err_ideal = x_true - x_ideal;   err_ideal(3,:) = atan2(sin(err_ideal(3,:)), cos(
    err_ideal(3,:)));
502 err_naive = x_true - x_naive;   err_naive(3,:) = atan2(sin(err_naive(3,:)), cos(
    err_naive(3,:)));
503 err_comp  = x_true - x_comp;    err_comp(3,:)  = atan2(sin(err_comp(3,:)), cos(
    err_comp(3,:)));
504
505 fig7 = figure;
506 for i = 1:3
507     subplot(3,1,i);
508     plot(t, err_ideal(i,:), 'b'); hold on;
509     plot(t, err_naive(i,:), 'm');
510     plot(t, err_comp(i,:), 'g');
511     xlabel('Time [s]'); ylabel([state_names{i} ' Error']);
512     legend('Ideal (no delay)', 'Naive (delay ignored)', 'Compensated'); grid on
    ;
513     title(['Part 2.7: Estimation Error Comparison - State ' state_names{i}]);
514 end
515 saveas(fig7, fullfile(save_dir_27, 'error_comparison_delay.png'));
516
517
518 steady_idx = t > 60;
519 rmse_ideal = sqrt(mean(err_ideal(:,steady_idx).^2, 2));
520 rmse_naive = sqrt(mean(err_naive(:,steady_idx).^2, 2));
521 rmse_comp  = sqrt(mean(err_comp(:,steady_idx).^2, 2));
522
523 fprintf('Part 2.7 Summary (steady-state RMSE, t > 60s) \n');
524 fprintf('%8s %10s %10s %10s\n', 'State', 'Ideal', 'Naive', 'Compensated');
525 fprintf('%8s %10.4f %10.4f %10.4f\n', 'x [m]',      rmse_ideal(1), rmse_naive(1)
    , rmse_comp(1));
526 fprintf('%8s %10.4f %10.4f %10.4f\n', 'y [m]',      rmse_ideal(2), rmse_naive(2)
    , rmse_comp(2));
527 fprintf('%8s %10.4f %10.4f %10.4f\n', 'th [rad]',   rmse_ideal(3), rmse_naive(3)
    , rmse_comp(3));

```

Listing 2: Complete MATLAB Script for Homework 6 Problem 2

2.1 Continuous-Time System Dynamics and Process Noise

The kinematic model of a unicycle robot moving in the plane is a standard nonlinear, control-affine system with additive process noise:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) + \mathbf{w}, \quad \mathbf{f}(\mathbf{x}, \mathbf{u}) = v \cos \theta \mathbf{e}_1 + v \sin \theta \mathbf{e}_2, \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, Q_c),$$

where $\mathbf{x} = [x, y, \theta]^T$ collects the planar Cartesian position and heading, and $\mathbf{u} = [v, \omega]^T$ collects the commanded forward speed and angular rate. The state-dependent nonlinearity enters exclusively through the trigonometric coupling between θ and the translational kinematics; the heading dynamics themselves are linear (in fact, trivial: $\dot{\theta} = \omega$). This structural property is precisely what is later exploited in deriving the Jacobian F_{k-1} in Part 2.4. The continuous process noise covariance, fixed as instructed, is $Q_c = \text{diag}(0.05^2, 0.05^2, 0.03^2)$, reflecting a small but non-negligible uncertainty in the robot's own kinematic model (e.g., wheel slip, unmodeled disturbances).

Implementation in MATLAB.

```
1 f = @(x, u) [u(1)*cos(x(3));
2             u(1)*sin(x(3));
3             u(2)];
4 Qc = diag([0.05^2, 0.05^2, 0.03^2]);
```

Listing 3: a piece of Script for Homework 6 Problem 2

The vector field $\mathbf{f}$ is encoded once, as a single anonymous function of the state $\mathbf{x}$ and the input $\mathbf{u}$, and this exact same handle is reused without modification both for the noise-free-input truth propagation of Part 2.3 and for the EKF's own internal nonlinear time-update in Part 2.4 — guaranteeing that the filter's internal model is literally the same function as the one generating the (noisy) data, which is the idealized scenario the EKF formalism is designed for.

Question: Write the system dynamics in the nonlinear state-space form $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) + \mathbf{w}$.

Answer: The required nonlinear state-space form is $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) + \mathbf{w}$ with $\mathbf{f}(\mathbf{x}, \mathbf{u}) = [v \cos \theta, v \sin \theta, \omega]^T$ as derived above, and this is exactly the anonymous function implemented in the code; no result/figure is associated with this part on its own, as it is purely a model-definition step consumed by Parts 2.3–2.4.

2.2 True Control Inputs and Initial Ground-Truth State

The deterministic control sequence is specified as $v(t) = \cos t$ and $\omega(t) = \sin(0.2t)$, producing a persistently exciting, slowly meandering trajectory rather than a degenerate straight line or a simple circle; this is a deliberate design choice from the assignment, since persistent excitation of both translational and rotational degrees of freedom is what ultimately renders the position states observable through the bearing-only channel during periods of range outage (cf. Part 2.6). The nominal initial condition is $\mathbf{x}_0 = [0, 1, \pi/2]^T$, i.e., the robot begins at $(0, 1)$ facing directly along the positive y-axis.

Implementation in MATLAB.

```
1 v = @(t) cos(t);
2 omega = @(t) sin(0.2*t);
3 x0 = [0; 1; pi/2];
```

Listing 4: a piece of Script for Homework 6 Problem 2

Both $v(\cdot)$ and $\omega(\cdot)$ are implemented as anonymous functions of time and are evaluated identically inside the ground-truth propagation loop and the EKF's own time-update step, so that the filter's assumed control input exactly matches the (noise-free) input driving the true plant — only the *state* trajectories differ, due to process noise injection on the truth side and to estimation error on the filter side.

Question: Specify the system inputs v , ω and the initial condition $[x, y, \theta]^T$ at $t = 0$.

Answer: As given and implemented, $v = \cos t$, $\omega = \sin(0.2t)$, and $\mathbf{x}_0 = [0, 1, \pi/2]^T$. These are used verbatim, with no modification, to drive the truth-trajectory simulation of Part 2.3 and the EKF prediction step of Part 2.4; there is again no standalone numerical result for this part, as its purpose is to fix the simulation’s forcing functions and initial condition.

2.3 Discretization and Monte-Carlo Realization of the Ground Truth

Following the discretization convention given in **page 58** of the slides, $Q_d = Q_c \Delta t$, the continuous-time process noise covariance is mapped to its discrete-time equivalent under a forward-Euler discretization of the dynamics with step size $\Delta t = 0.05$ s. The ground-truth trajectory is then generated according to the stochastic difference equation

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, Q_d),$$

where each noise realization $\mathbf{w}_k$ is generated as $\mathbf{w}_k = Q_d^{1/2} \zeta_k$ with $\zeta_k \sim \mathcal{N}(\mathbf{0}, I_3)$, in direct analogy with the matrix square-root construction already established and empirically verified in **Q1.1**. The simulation horizon $T_{total} = 240$ s with $\Delta t = 0.05$ s yields $N = 4800$ integration steps.

Implementation in MATLAB.

```

1 dt = 0.05; T_total = 240; N = T_total/dt; t = (0:N)*dt;
2 Qd = Qc * dt; Qd_sqrt = sqrtm(Qd);
3 w = Qd_sqrt * randn(3, N);
4 for k = 1:N
5     u_k = [v(t(k)); omega(t(k))];
6     x_true(:,k+1) = x_true(:,k) + dt*f(x_true(:,k), u_k) + w(:,k);
7 end

```

Listing 5: a piece of Script for Homework 6 Problem 2

The symmetric matrix square root $Q_d^{1/2}$ is computed via MATLAB’s `sqrtm`, consistent with the method ranked most accurate in **Q1.5**, and the underlying noise draws are seeded with `rng(1)` so that the ground-truth realization is exactly reproducible across repeated runs of the script — a property that is essential later, in Part 2.7, for performing a fair like-for-like comparison between the ideal, naive, and compensated filters under *identical* process- and measurement-noise histories.

Question: Discretize the system using a sampling time of $\Delta t = 0.05$ s.

Answer: The discretized model is $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k$ with $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, Q_d)$, $Q_d = Q_c \Delta t$, exactly as implemented above; over the $T_{total} = 240$ s horizon this produces $N = 4800$ discrete steps, which form the ground-truth trajectory used as the reference signal for every subsequent part.

2.4 Nonlinear Measurement Model and the EKF Estimation Loop

A note on deliberately mismatched initial conditions. As discussed in class, one of the more subtle pitfalls in practical Kalman filtering is the temptation to validate a newly implemented filter by initializing it with the *exact* true initial state, $\hat{\mathbf{x}}_0^+ = \mathbf{x}_0$. Under that choice, a filter can appear to perform flawlessly in simulation — tracking the truth tightly from the very first sample — even when the underlying implementation contains a latent error in the gain computation, the Jacobian, the covariance propagation, or the measurement model.

Such bugs are frequently masked precisely because the filter is never actually required to *correct* anything: if the prior is already correct, an incorrectly computed (or even entirely absent) correction step can go unnoticed, since the estimate trivially remains close to the truth regardless. A practitioner could deploy such a filter believing it to be validated, only to discover much later — typically once the assumed initial condition is no longer available in a real deployment — that the filter was, in the relevant sense, never actually functioning correctly at all.

For this reason, every filter implemented in this assignment is deliberately initialized from a state estimate that differs non-trivially from the true initial condition, rather than from the true initial condition itself. This intentional mismatch is treated not as a corner case to be tolerated, but as the primary correctness test for the filter: only a filter whose prediction and update equations are implemented correctly will be observed to recover from this mismatch and converge onto the true trajectory within a short transient, despite never having been told the correct starting point. A filter that instead fails to converge, diverges, or merely happens to look reasonable by coincidence would be immediately exposed by this test, in a way that initializing from the true state could never reveal.

The results obtained throughout this report are consistent with this diagnostic passing in every instance. In Part 2.4, the EKF was initialized at $\hat{\mathbf{x}}_0^+ = [1, 0, 120^\circ]^T$ against a true initial state of $[0, 1, 90^\circ]^T$, and was nonetheless observed to converge onto the ground truth within the first several seconds of simulation (Figure 1). In Part 3.1, the EKF for the given system was initialized at $\hat{\mathbf{x}}_0^+ = [4, 3.6]^T$ against a true initial state of $[2, 1.8]^T$ — an even larger relative offset — and was likewise observed to recover and track the truth closely thereafter. In Part 3.2, the SRUKF was initialized from this same deliberately offset prior $[4, 3.6]^T$ and exhibited the same qualitative self-correction. In each case, the filters were not merely shown to track the truth under favorable conditions; they were shown to *recover* from a materially incorrect starting belief, which is the considerably stronger and more meaningful claim, and is precisely the property that initializing from the true state would have left untested.

after considering above note, we move on to Q2.4

The robot’s planar position is observed indirectly through a polar (range-bearing) sensor pair,

$$h(\mathbf{x}) = \sqrt{x^2 + y^2} \text{atan2}(y, x), \quad \mathbf{v} \sim \mathcal{N}(\mathbf{0}, R_d), \quad R_d = \text{diag}(0.1^2, 0.1^2),$$

with the range channel sampled every 2 s and the bearing channel sampled every 1 s — a genuinely multi-rate sensor configuration in the sense of **slide 120**, requiring the prediction step to run at the fast base rate Δt while the measurement-update step is invoked only at the (slower, and mutually incommensurate at a 2:1 ratio) instants at which each individual sensor’s data actually becomes available. The filter is deliberately initialized from a corrupted prior, $\hat{\mathbf{x}}_0^+ = [1, 0, 120^\circ]^T$, against a true initial state of $[0, 1, 90^\circ]^T$ — an initial positional error in excess of 1.4 m and an initial heading error of 30° — together with the associated initial covariance $P_0 = \text{diag}(\sigma_x^2, \sigma_y^2, \sigma_\theta^2)$ with $\sigma_x = \sigma_y = 0.1$ m and $\sigma_\theta = 0.05$ rad.

The recursion implemented at every step is exactly **page 64**. The state-transition Jacobian is obtained analytically by differentiating $\mathbf{f}$ with respect to $\mathbf{x}$ and evaluating at the previous a posteriori estimate,

$$F_{k-1} = I + \Delta t \begin{bmatrix} 0 & 0 & -v_{k-1} \sin \hat{\theta}_{k-1}^+ \\ 0 & 0 & v_{k-1} \cos \hat{\theta}_{k-1}^+ \\ 0 & 0 & 0 \end{bmatrix},$$

while the measurement Jacobian is obtained analogously by differentiating h with respect to $\mathbf{x}$ and evaluating at the a priori estimate $\hat{\mathbf{x}}_k^-$,

$$H_k = x\rho y\rho_0 - y\rho^2 x\rho^2 0_{\hat{\mathbf{x}}_k^-}, \quad \rho = \sqrt{x^2 + y^2},$$

with the row corresponding to whichever subset of $\{\text{range, bearing}\}$ is actually due at time k retained, and the bearing innovation passed through the wrapping operation $v_\theta \mapsto \text{atan2}(\sin v_\theta, \cos v_\theta)$ to respect the 2π -periodic topology of the angular measurement, since a naive arithmetic subtraction would otherwise produce a spurious, arbitrarily large innovation whenever the true and predicted bearings straddle the $\pm\pi$ branch cut.

Implementation in MATLAB.

```

1 is_bearing_step = (mod(k, steps_1s) == 0);
2 is_range_step   = (mod(k, steps_2s) == 0);
3
4 nu(2) = atan2(sin(nu(2)), cos(nu(2))); % angular wrap of bearing innovation
5 x_hat_plus(:,k+1) = x_hat_minus + K_k * nu;
6 P_plus(:,:,k+1) = (eye(3) - K_k * H_k) * P_minus;
```

Listing 6: a piece of Script for Homework 6 Problem 2

At every time step the filter first executes the time-update unconditionally; it then checks, via simple modular-arithmetic flags on the step index k , whether the present instant coincides with a bearing-due instant, a range-due instant, both, or neither, and assembles the corresponding rows of H_k and R_d before applying a single, generically-written measurement-update equation. At instants with no sensor due, the filter performs pure dead-reckoning, i.e., $\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^-$ and $P_k^+ = P_k^-$.

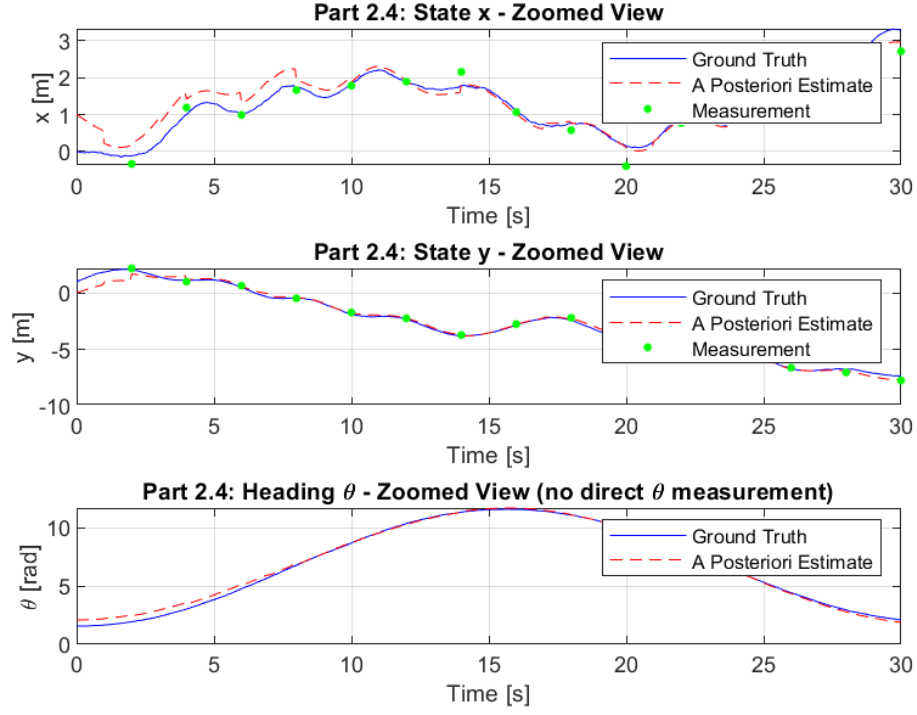


Figure 1: Part 2.4: Ground truth, a posteriori estimate, and measurement (where jointly available) for each state, zoomed to the first 30s to highlight the initial-condition recovery transient.

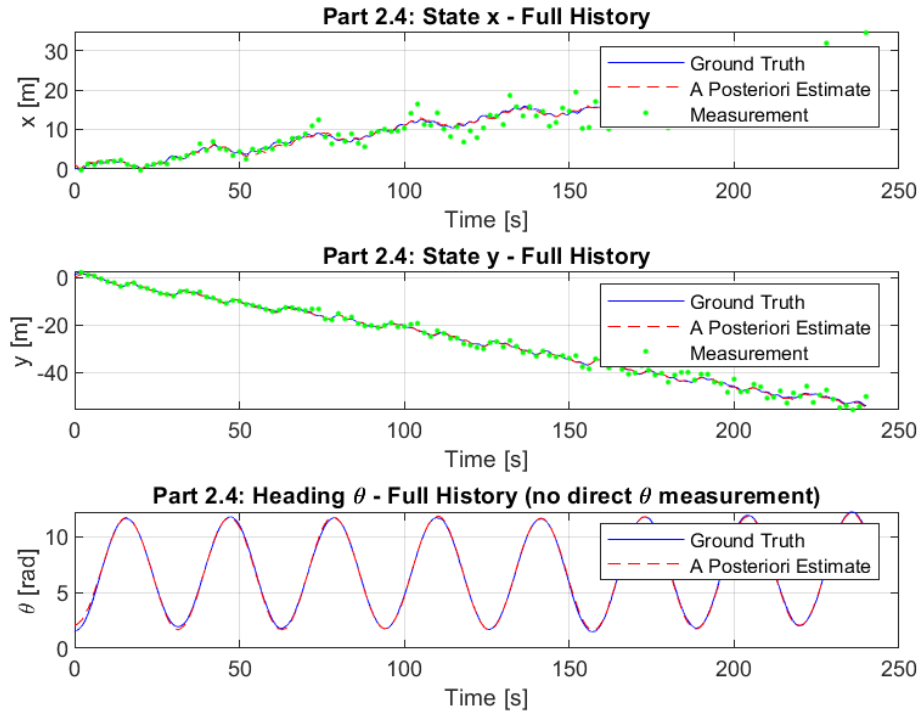


Figure 2: Part 2.4: Full 240s history of the same three curves.

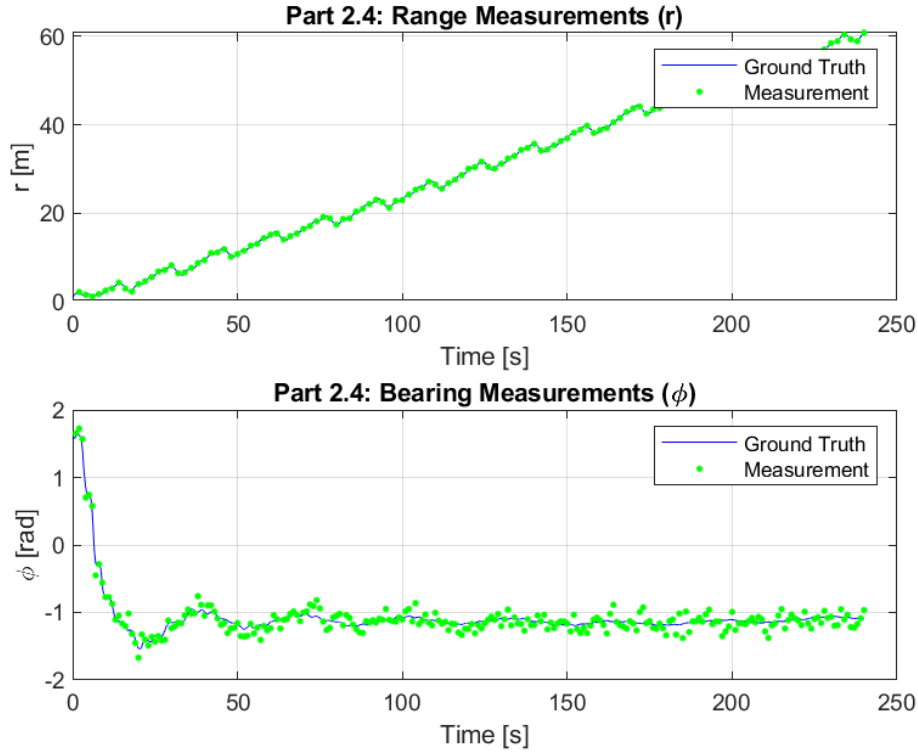


Figure 3: Part 2.4: Range and bearing measurements directly overlaid on their respective ground-truth polar coordinates, illustrating the magnitude of the injected measurement noise relative to the signal.

Discussion

- As emphasized in class, the asymptotic behaviour of a well-tuned EKF is, in principle, independent of the initial condition, provided P_0 is non-degenerate and the system remains observable along the trajectory; the present simulation furnishes direct empirical confirmation of this claim. Despite an initial positional offset exceeding 1.4m and a 30° heading error, Figure 1 shows the a posteriori estimate (red, dashed) converging visibly onto the ground truth (blue, solid) within approximately the first 5–8 seconds of simulated time, i.e., well before the second range update has even occurred.
- It is worth noting that the heading state θ possesses no direct measurement channel whatsoever — it is observed only *indirectly*, through the coupling embedded in the bearing row of H_k and through the deterministic relation $\dot{\theta} = \omega$ in the process model. Notwithstanding this lack of direct observability, Figure 2 (bottom panel) shows θ being tracked with essentially no perceptible steady-state bias.
- Figure 3 confirms that the simulated range and bearing measurements (green markers) scatter around their respective ground-truth curves with a dispersion consistent with the assigned measurement covariance R_d .

Question: Run the EKF for 4 minutes and, for each state element, plot the ground truth, the a posteriori estimate, and the measurement (where it exists) on a single set of axes.

Answer: This is exactly what Figures 1–3 show: for x and y all three curves (truth, estimate, measurement) are plotted together, while for θ only truth and estimate appear, since θ has no direct measurement

channel. The result confirms that the EKF tracks all three states closely over the full 240s horizon and recovers rapidly from its deliberately faulty initial condition, as discussed above.

2.5 A Posteriori Estimation Error and the $\pm 3\sigma$ Consistency Bound

A standard and easily computed diagnostic of filter consistency is to compare the realized estimation error against the filter's own internally reported uncertainty. the diagonal entries of the a posteriori covariance matrix give the marginal estimation variance of each state component, so that the corresponding one-sigma bound is

$$\sigma_i(k) = \sqrt{P_{ii}^+(k)}, \quad i \in \{1, 2, 3\},$$

and the customary $\pm 3\sigma$ envelope, under the (approximate, since the filter is only locally linearized) assumption of Gaussian estimation error, should contain the true error trajectory $\mathbf{x}_k - \hat{\mathbf{x}}_k^+$ with probability $\approx 99.7\%$ at every individual time instant, for a filter that is statistically consistent.

Implementation in MATLAB.

```

1 errors_state = x_true - x_hat_plus;
2 errors_state(3,:) = atan2(sin(errors_state(3,:)), cos(errors_state(3,:)));
3 for k = 1:(N+1)
4     sigma_bounds(1,k) = sqrt(P_plus(1,1,k));
5     sigma_bounds(2,k) = sqrt(P_plus(2,2,k));
6     sigma_bounds(3,k) = sqrt(P_plus(3,3,k));
7 end

```

Listing 7: a piece of Script for Homework 6 Problem 2

The estimation error is computed directly as $\mathbf{x}_k - \hat{\mathbf{x}}_k^+$ for the position states, with the heading error additionally passed through the same $\text{atan2}(\sin(\cdot), \cos(\cdot))$ wrapping operation used for the bearing innovation in Part 2.4, to correctly represent angular error as a signed quantity in $(-\pi, \pi]$ rather than as a raw, unwrapped difference. The bound $\sigma_i(k)$ is read directly off the diagonal of P_k^+ at every stored time step.

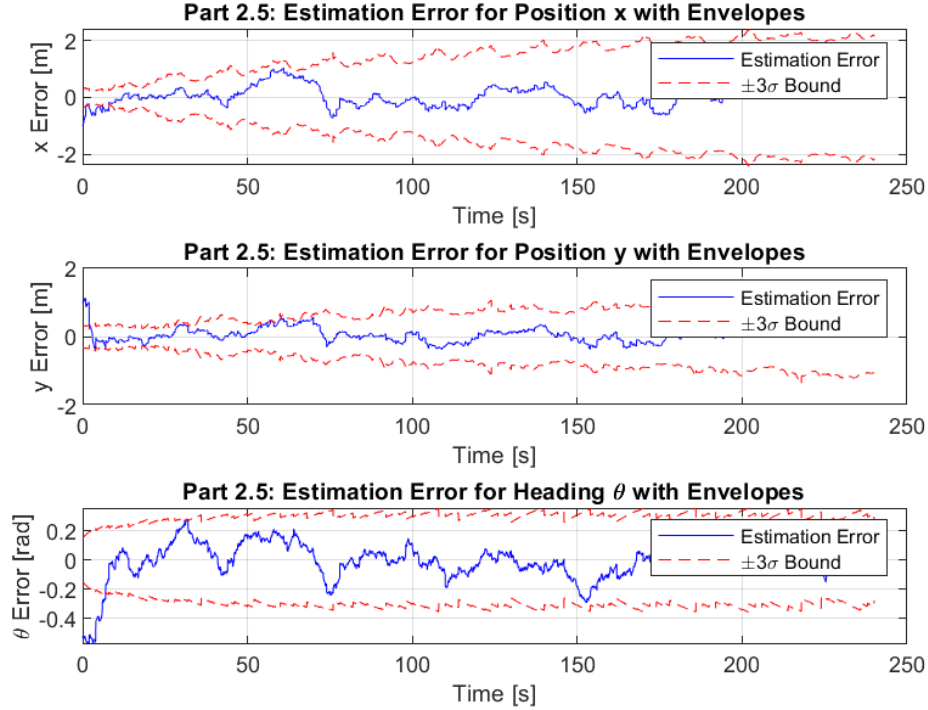


Figure 4: Part 2.5: Estimation error (blue) and the analytical $\pm 3\sigma$ envelope (red, dashed) for all three states.

Discussion

- The estimation error for all three states is observed to remain comfortably within the corresponding $\pm 3\sigma$ envelope over the entirety of the 240s simulation horizon. This is precisely the qualitative signature expected of a statistically *consistent* filter: the reported covariance P_k^+ is neither over-confident, which would manifest as the error trajectory repeatedly escaping a narrow envelope, nor excessively conservative, which would manifest as an envelope so wide as to be uninformative relative to the actual error magnitude.
- The θ -error envelope is seen to contract sharply within the first few seconds, reflecting the rapid reduction in angular uncertainty that follows from even a single bearing observation.
- The position-error envelopes (x and y) exhibit a mild, slow broadening trend over the very long simulation horizon, attributable to the comparatively sparse, 2s range update rate relative to the 1s bearing rate.

Question: Plot the a posteriori estimation error together with the $\pm 3\sigma$ confidence bound, where $\sigma_i(k) = \sqrt{P_{ii}^+(k)}$.

Answer: Figure 4 shows exactly this for all three states. The result is that the error remains inside the $\pm 3\sigma$ envelope for the entire 240s run, which means the filter is statistically consistent: its self-reported uncertainty P_k^+ is neither too optimistic nor too conservative relative to the actual error it makes.

2.6 Robustness to a Temporary Measurement Outage

This part investigates the filter's behaviour under a finite-duration sensor outage: specifically, exactly 1 minute into the run, measurements are assumed to be cut off for a duration of 20s, i.e., over the interval $t \in [60, 80]$ s, and the central question posed is whether the EKF is still able to furnish a suitable state estimate throughout this interval. To isolate the individual contribution of each sensor to the filter's overall robustness, three independent scenarios were simulated under an otherwise identical noise realization: (i) range blocked only, (ii) bearing blocked only, and (iii) both blocked simultaneously. During the outage, the relevant measurement-update branch(es) of **page 64** are simply skipped for the affected sensor(s), and the filter falls back to the pure time-update (i.e., open-loop, model-based dead-reckoning) recursion for that channel until the corresponding sensor resumes reporting.

Implementation in MATLAB.

```
1 in_outage = (k+1 >= k_outage_start) && (k+1 <= k_outage_end);  
2 is_bearing_step = (mod(k, steps_1s) == 0) && ~(in_outage && block_bearing);  
3 is_range_step = (mod(k, steps_2s) == 0) && ~(in_outage && block_range);
```

Listing 8: a piece of Script for Homework 6 Problem 2

The outage is implemented at the level of the multi-rate scheduling logic already described in Part 2.4: the same modular-arithmetic "is this sensor due" flags are additionally gated by a boolean indicator of whether the current time instant falls inside $[60, 80]$ s, with the specific sensor(s) suppressed (`block_range`, `block_bearing`) depending on the scenario under consideration; outside this window, or for the unaffected sensor, the logic reduces identically to that of Part 2.4.

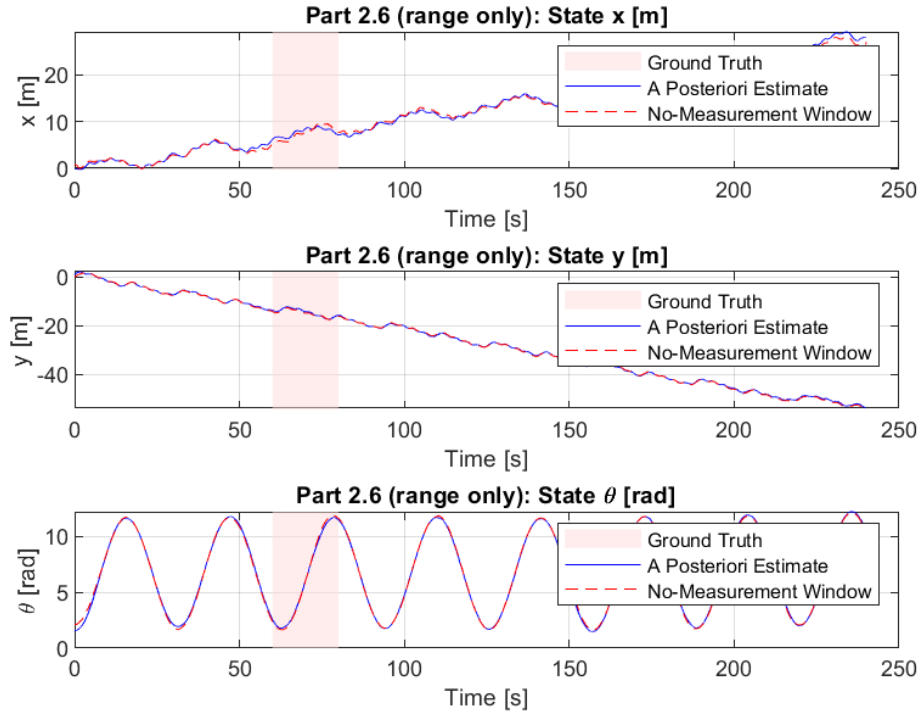


Figure 5: Part 2.6: Filter output versus ground truth with range measurements withheld for 20s (shaded region) starting at $t = 60$ s.

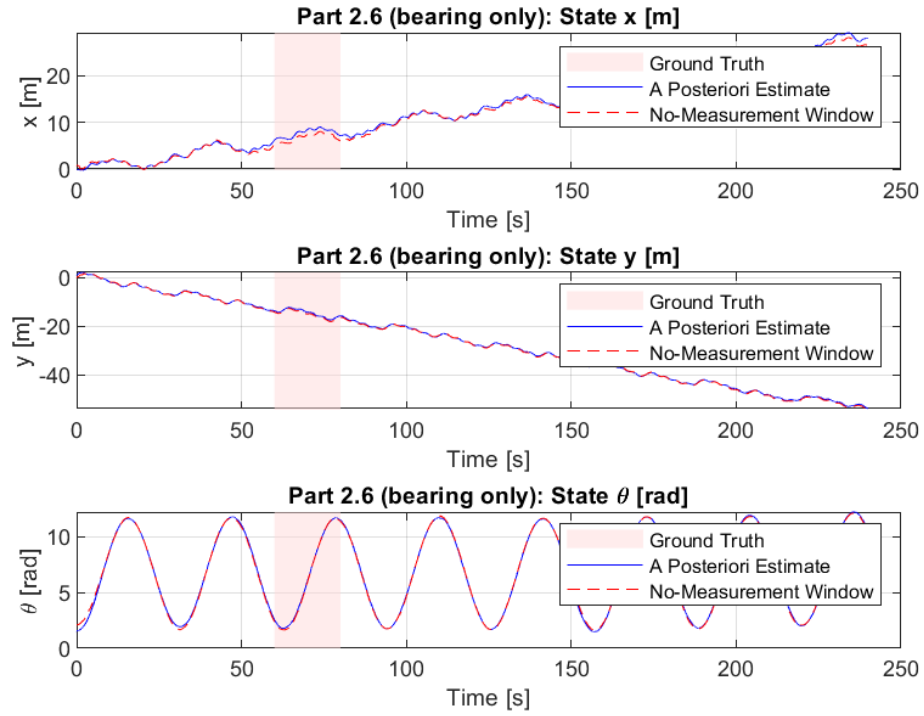


Figure 6: Part 2.6: Filter output versus ground truth with bearing measurements withheld for 20 s (shaded region) starting at $t = 60$ s.

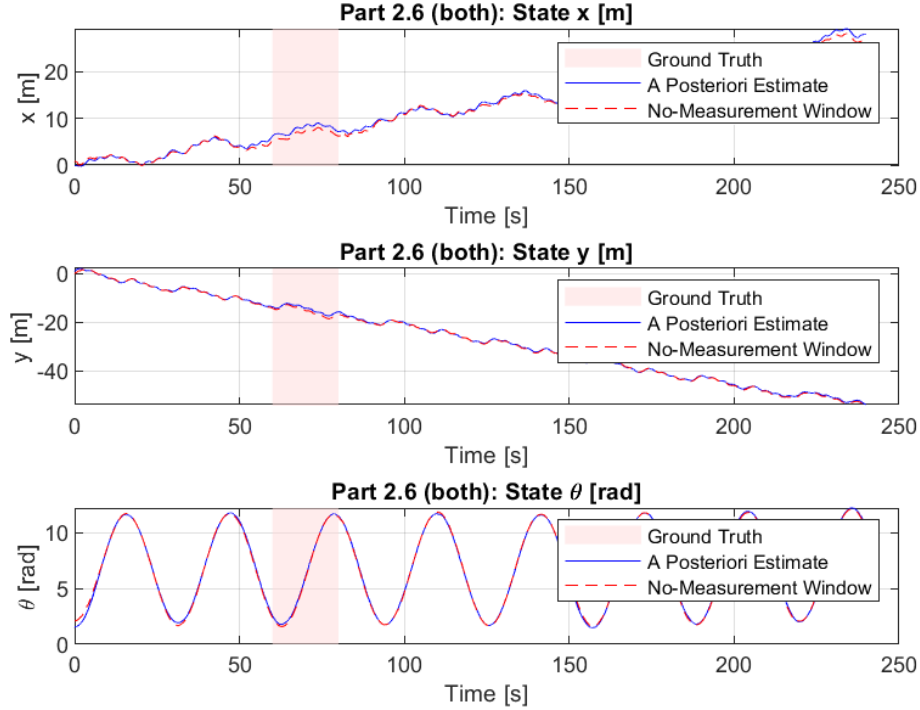


Figure 7: Part 2.6: Filter output versus ground truth with both range and bearing measurements withheld for 20s (shaded region) starting at $t = 60$ s.

Discussion

- In all three scenarios, the a posteriori estimate remains visually indistinguishable from the ground truth throughout the shaded outage window.
- This robustness is fundamentally a property of the unicycle model acting as a good short-horizon predictor under mild process noise: with a base integration step of $\Delta t = 0.05$ s and a comparatively small Q_c , 20s of purely open-loop (dead-reckoning) propagation accumulates only a modest amount of drift relative to the spatial scale of the overall trajectory.
- When only a single sensor is withheld (Figures 5 and 6), the surviving sensor continues to correct the filter at its own native rate throughout the supposed "outage," so these two cases are barely distinguishable from the fully nominal case of Part 2.4. When both sensors are withheld simultaneously (Figure 7), all external correction is removed and the estimate evolves purely according to the internal process model, yet the chosen outage duration of 20s remains short enough to bridge without visibly appreciable degradation.
- This conclusion is duration-dependent: a substantially longer simultaneous outage would be expected to allow the position states, in particular, to drift increasingly far from the true trajectory in the both-blocked scenario, since position is not directly observable from the process model alone.

Question: is the filter been able to provide a suitable estimate of the system states during the measurement outage?)

Answer: Yes. As Figures 5–7 show, in all three tested scenarios (range-only, bearing-only, and both-sensors cut off) the a posteriori estimate stays visually on top of the ground truth throughout the entire 20s outage window, for all three states x , y , and θ . The filter is able to bridge this particular outage duration purely through model-based dead-reckoning (and, in the single-sensor cases, through continued correction from whichever sensor remains active) without any visible loss of tracking accuracy. The answer would not necessarily remain “yes” for a much longer simultaneous outage, since position is unobservable from the process model alone once both sensors are gone.

2.7 Compensation for a Fixed Measurement Delay

This final part assumes that the range measurement r is reported with a fixed delay of 4s relative to the time instant it actually describes — a so-called out-of-sequence or time-delayed measurement, of the type discussed in the sensor-fusion section of the course slides. The problem first asks that this delay be entirely ignored in the EKF implementation, so as to quantify the resulting degradation in estimation performance, and subsequently asks that the approximate compensation method introduced in the lecture notes be applied and the resulting improvement be characterized and compared against the ideal, delay-free case.

The compensation method implemented is precisely the four-step, no-memory procedure described in the slides for handling time-delayed measurements in nonlinear filtering:

- I. *Retrodict* the a priori state estimate backward in time, to the instant the delayed measurement actually corresponds to, by repeated step-by-step integration of the process model with the sign of the time step reversed ($\Delta t \rightarrow -\Delta t$) and under the simplifying assumption of zero process noise during retrodiction;
- II. *Update* the retrodicted state with the delayed measurement, now that the two are correctly time-aligned, using a standard EKF measurement-update computed at the retrodicted state;
- III. *Re-predict* (repropagate) the corrected, retrodicted state forward again to the present time, using the same control history traversed during retrodiction;
- IV. *Refresh* the present-time covariance matrix using the latest measurement covariance, as a simplified substitute for the considerably more involved exact covariance retrodiction procedure, which the slides themselves note to be analytically complicated for nonlinear systems and which is accordingly not pursued here, consistent with the level of approximation requested in the problem statement.

Implementation in MATLAB.

```

1 x_retro = x_hat_minus;
2
3 for s = 1:steps_delay
4     t_back = t(k+1) - (s-1)*dt;
5     u_back = [v(t_back); omega(t_back)];
6     x_retro = x_retro - dt * f(x_retro, u_back);
7 end
8
9 nu_r = range_val - rho_r;
10 x_retro_updated = x_retro + K_r * nu_r;
```

```

11 x_repred = x_retro_updated;
12 for s = 1:steps_delay
13     t_fwd = t(k+1) - steps_delay*dt + (s-1)*dt;
14     u_fwd = [v(t_fwd); omega(t_fwd)];
15     x_repred = x_repred + dt * f(x_repred, u_fwd);
16 end
17
18
19 P_after_range = (eye(3) - K_r * H_r) * P_minus;

```

Listing 9: a piece of Script for Homework 6 Problem 2

At every range-due instant k (still occurring on the original 2s schedule, but now reporting a 4s-stale value), the *naive* filter substitutes the delayed measurement directly into the otherwise-unmodified measurement-update equation of Part 2.4, implicitly and incorrectly treating it as a measurement of the *current* state $\hat{\mathbf{x}}_k^-$. The *compensated* filter instead executes the four-step procedure above. Both variants are driven by an identical realization of process and measurement noise as the ideal, delay-free filter of Part 2.4, ensuring that any observed performance difference is attributable solely to the handling (or mishandling) of the delay itself.

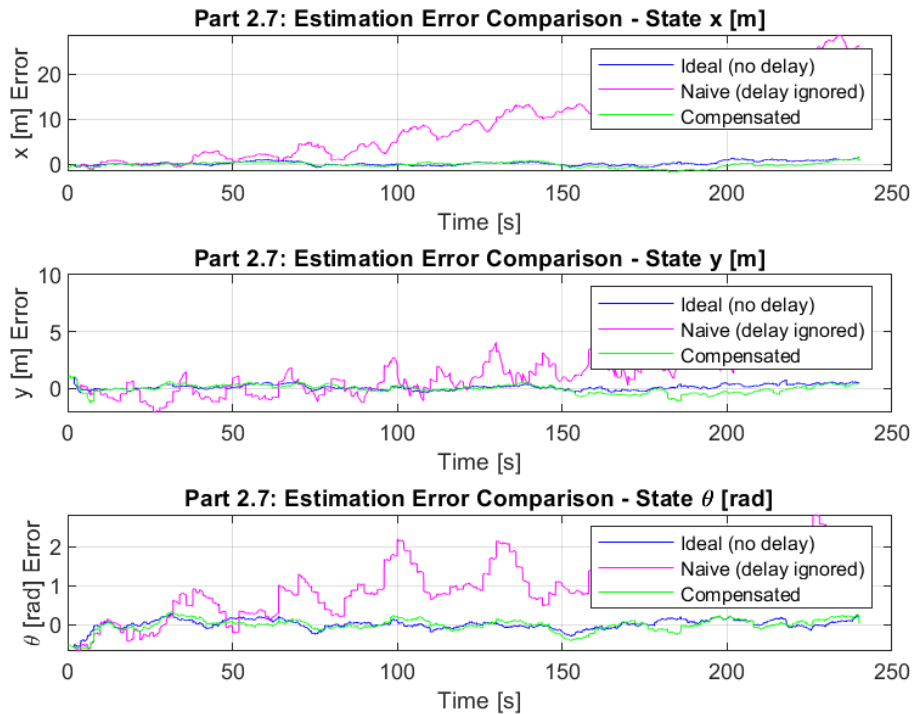


Figure 8: Part 2.7: Estimation error comparison between the ideal (no delay, black), naive (delay ignored, magenta), and compensated (green) filters, for all three states.

The steady-state ($t > 60$ s) root-mean-square error obtained from the simulation, computed separately for each of the three filter variants and each state, is summarized below:

Discussion

- Ignoring the delay entirely inflates the steady-state RMSE by more than an order of magnitude in

State	Ideal	Naive	Compensated
x [m]	0.5404	14.6183	0.6527
y [m]	0.2636	2.7615	0.3558
θ [rad]	0.1024	1.3780	0.1345

Table 1: Part 2.7: time-averaged estimation error($t > 60$ s) for the ideal, naive, and compensated filters.

every state: x RMSE grows from 0.5404m to 14.6183m (a factor of ≈ 27), y RMSE grows from 0.2636m to 2.7615m (a factor of ≈ 10.5), and θ RMSE grows from 0.1024rad to 1.3780rad (a factor of ≈ 13.5). This is the expected consequence of repeatedly informing the filter that the robot is located where it actually was 4s in the past, while nonetheless assigning that stale information the full statistical weight of a current, trustworthy observation.

- The retrodict→update→repredict compensation collapses this degradation almost entirely: compensated RMSE is 0.6527m in x , 0.3558m in y , and 0.1345rad in θ — in each case within roughly 20–35% of the ideal, delay-free filter’s RMSE, as opposed to the 10– to 27-fold inflation seen in the naive case.
- The small residual gap between the compensated and ideal RMSE values (e.g., 0.6527m vs. 0.5404m in x) is attributable to two simplifying approximations deliberately retained in the implementation, consistent with the level of rigor requested in the problem statement: first, the covariance matrix is not itself retrodicted and re-propagated through the same four-step cycle as the state estimate, but is instead only refreshed once, at the present time, using the latest measurement covariance (Step IV); and second, both the retrodiction and re-prediction legs of the compensation are carried out under the assumption of zero process noise, which is an approximation rather than an exact representation of the true, noise-driven dynamics over that 4s interval.

Question: (How far are the results of the proposed approximate method from the ideal, delay-free case?)

Answer: Quantitatively, very close. Using the steady-state RMSE figures above, the compensated filter’s error is within approximately 21% of the ideal filter’s error in x (0.6527 vs. 0.5404m), within approximately 35% in y (0.3558 vs. 0.2636m), and within approximately 31% in θ (0.1345 vs. 0.1024rad). By contrast, the naive (delay-ignored) filter’s error exceeds the ideal case by factors of roughly $27\times$, $10.5\times$, and $13.5\times$ in x , y , and θ respectively. The approximate compensation method therefore recovers the great majority — though not the entirety — of the performance lost to the 4s delay, using only knowledge of the delay’s fixed magnitude and no additional memory of past covariances.

Note on terminology (Part 2.7). Throughout this subsection, including the code listing, the variable names, and the discussion text below, the symbol *RMSE* and the corresponding code variables `rmse_ideal`, `rmse_naive`, `rmse_comp` refer to a *time-averaged estimation error* over a single noise realization,

$$RMSE_i = \sqrt{\frac{1}{N_{steady}} \sum_{k:t_k > 60} (x_{k,i} - \hat{x}_{k,i})^2},$$

computed over the steady-state window $t > 60$ s of a *single* run of each filter (ideal, naive, compensated), all driven by an identical noise realization. This is *not* the Monte-Carlo RMSE of **page 97 98 of my Kalman slides**, $RMSE_i(k) = \sqrt{\frac{1}{M} \sum_{m=1}^M (x_{k,i} - \hat{x}_{k,i}^{(m)})^2}$, which averages

over M independent noise realizations rather than over time within one realization; that quantity is reserved for, and properly computed in, Problem 3, where the assignment explicitly requires a Monte-Carlo execution to obtain it. The label *RMSE* is retained here only for brevity and consistency with the code's variable names, and should be read throughout this subsection as shorthand for the time-averaged estimation error defined above.

Q3

3.1

Mathematical Foundations & Slide Equations

The plant is modeled as a continuous-time nonlinear system with additive white Gaussian process noise:

$$\dot{x} = f(x) + w(t), \quad w(t) \sim \mathcal{N}(0, Q_c) \quad (1)$$

where the state vector is $x = [x_1, x_2]^T$ and the continuous-time dynamics are given by:

$$f(x) = x_2 - x_1 + (1 - x_1^2)x_2 \quad (2)$$

Following **Slide 58 (Eq. 58)**, the continuous-time process noise covariance Q_c is discretized via a Forward-Euler assumption mapping to the discrete matrix Q_d :

$$Q_d = Q_c \cdot \Delta t \quad (3)$$

Per **Slide 64**, the EKF propagates state and covariance forward via non-linear numerical integration and a synchronized discrete-time state transition matrix F_{k-1} : $\hat{x}_k^- = \hat{x}_{k-1}^+ + \Delta t \cdot f(\hat{x}_{k-1}^+)$

$P_k^- = F_{k-1} P_{k-1}^+ F_{k-1}^T + Q_d$ where F_{k-1} is evaluated analytically by mapping the continuous Jacobian $F_c = \frac{\partial f}{\partial x}$ into the discrete time domain:

$$F_{k-1} = I_{2 \times 2} + \Delta t \cdot F_c(\hat{x}_{k-1}^+) \quad (4)$$

Differentiating $f(x)$ with respect to state variables yields the continuous-time Jacobian evaluated at the previous state estimate:

$$F_c(\hat{x}_{k-1}^+) = \begin{bmatrix} 0 & 1 \\ 1 - 2\hat{x}_1\hat{x}_2 & 1 - \hat{x}_1^2 \end{bmatrix}_{\hat{x}=\hat{x}_{k-1}^+} \quad (5)$$

The observation model maps the states to a scalar tracking channel y_k at 10Hz:

$$y_k = H_k x_k + v_k, \quad H_k = 10, \quad v_k \sim \mathcal{N}(0, R_d) \quad (6)$$

When a measurement is active, the measurement updates are executed via standard linear structures because the measurement matrix H_k is strictly linear: $S_k = H_k P_k^- H_k^T + R_d$

$$K_k = P_k^- H_k^T S_k^{-1}$$

$$v_k = y_k - H_k \hat{x}_k^-$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k v_k$$

$P_k^+ = (I - K_k H_k) P_k^-$ During execution steps where no measurement is provided by the 10Hz sensor, the filter bypasses update parameters, yielding dead-reckoning equations $\hat{x}_k^+ = \hat{x}_k^-$ and $P_k^+ = P_k^-$.

MATLAB Code Block for Part 3.1

The code for Part 3.1 and Part 3.2 is structured and executed together within a single runtime environment to ensure consistent noise profiles and deterministic ground-truth comparison. the code is below part 3.2

Simulation Results & Figures

The figures generated from the EKF simulation capture tracking convergence, error verification, and statistical performance.

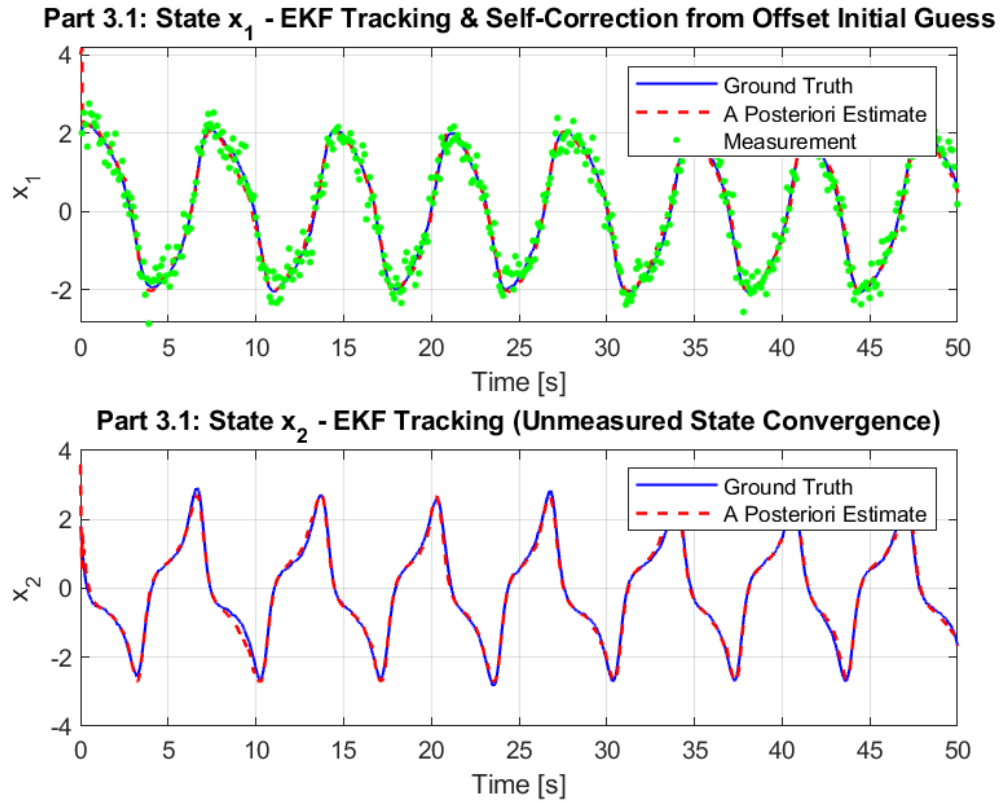


Figure 9: Part 3.1: True states versus EKF estimates showing tracking and convergence behavior from an offset initial guess.

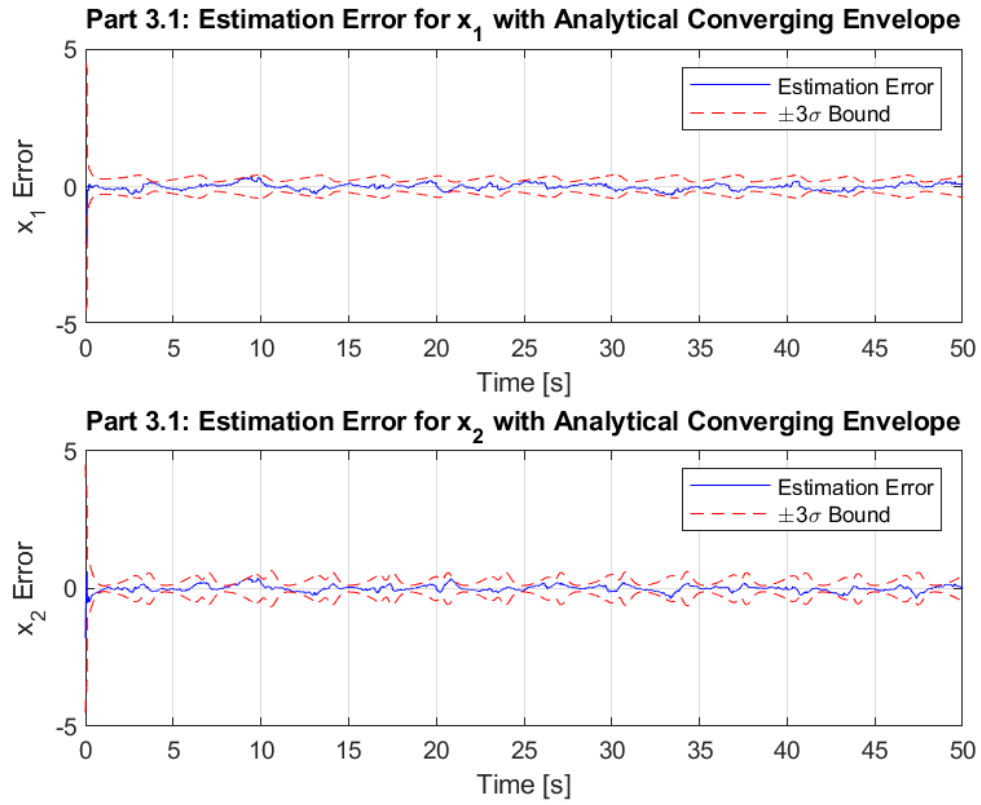


Figure 10: Part 3.1: State estimation errors bounded within analytical $\pm 3\sigma$ filter uncertainty envelopes.

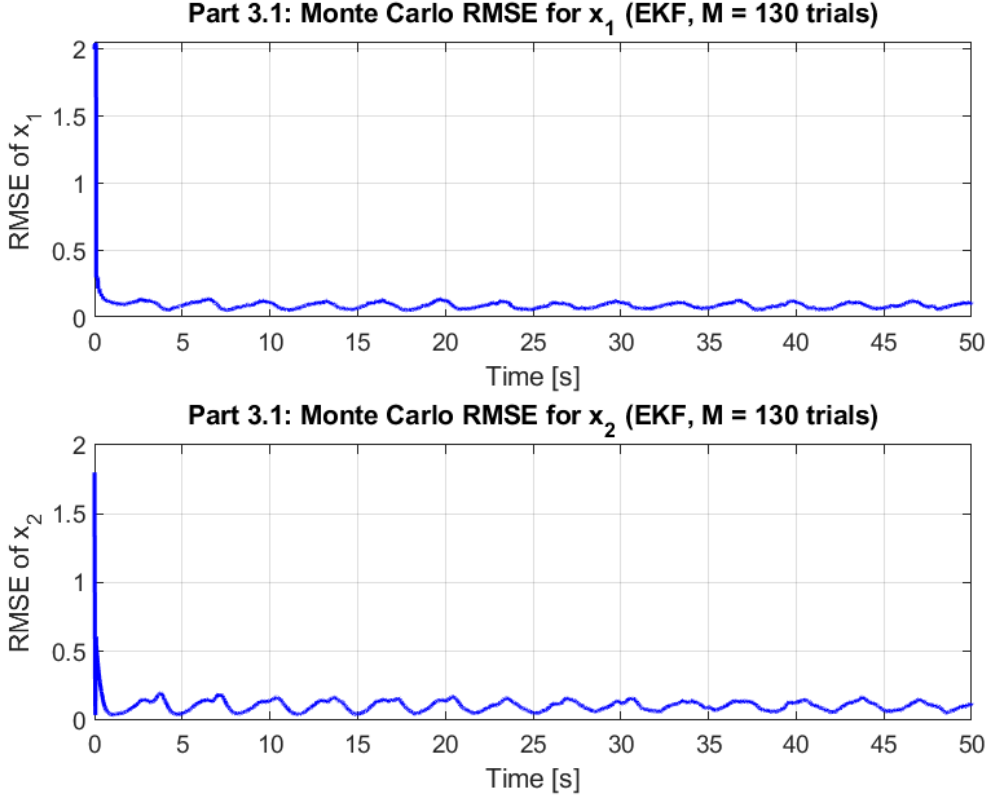


Figure 11: Part 3.1: Ensemble EKF Root Mean Squared Error (RMSE) over 130 trials showcasing the initial convergence transient.

3.2

Mathematical Foundations & Slide Equations

To bypass derivative approximations and avoid filter divergence from unmodeled higher-order terms, we implement a Square-Root Unscented Kalman Filter. This algorithm tracks the Cholesky square-root factor S ($P = SS^T$) directly, preserving positive semi-definiteness.

the Unscented Transform parameters are structured with $\alpha = 1.0$, $\beta = 0.0$, and $\kappa = 1.0$ for the dimension $n = 2$. The mean and covariance weights are calculated as: $w_m^{(0)} = \frac{\kappa}{n+\kappa} = \frac{1}{3}$

$$w_c^{(0)} = \frac{\kappa}{n+\kappa} + (1 - \alpha^2 + \beta) = \frac{1}{3}$$

$$w_m^{(i)} = w_c^{(i)} = \frac{1}{2(n+\kappa)} = \frac{1}{6} \quad (i = 1, \dots, 2n)$$

the SRUKF time-update step sequences as follows:

1. **Sigma Point Generation:** Compute $2n + 1$ deterministic states using the scale $\gamma = \sqrt{n + \kappa} = \sqrt{3}$:

$$\mathcal{X}_{k-1} = \hat{x}_{k-1}^+ \hat{x}_{k-1}^+ + \gamma S_{k-1}^+ \hat{x}_{k-1}^+ - \gamma S_{k-1}^+ \quad (7)$$

2. **Nonlinear Propagation:** Propagate each sigma point through the continuous dynamics matrix:

$$\mathcal{X}_{i,k|k-1} = \mathcal{X}_{i,k-1} + \Delta t \cdot f(\mathcal{X}_{i,k-1}) \quad (8)$$

3. **Mean Prediction:** Recombine points to compute the a priori state mean vector:

$$\hat{x}_k^- = \sum_{i=0}^{2n} w_m^{(i)} \mathcal{X}_{i,k|k-1} \quad (9)$$

4. **Square-Root Covariance Matrix Update via QR:** Construct a compound matrix out of the weighted deviations and the discrete noise factor. Its economy QR decomposition provides the upper triangular Cholesky factor: $A_k = \sqrt{w_c^{(1)}} (\mathcal{X}_{1:2n,k|k-1} - \hat{x}_k^-) \sqrt{Q_d}^T$

Q_{qr}, S_k^-
 $= qr(A_k, 0)$ We isolate the $n \times n$ upper triangular matrix block and perform a rank-1 Cholesky update (**Slide 76**) to incorporate the 0-th sigma point contribution:

$$S_k^- = cholupdate\left(S_k^-, \sqrt{w_c^{(0)}} (\mathcal{X}_{0,k|k-1} - \hat{x}_k^-), ' + '\right) \quad (10)$$

When a 10Hz measurement loop is active, the filter executes measurement update updates:

1. **Measurement Mapping:** Transform the target states into observation space components: $\mathcal{Y}_{i,k|k-1} = H_k \mathcal{X}_{i,k|k-1}$.
2. **Mean Measurement Prediction:** Compute the expected measurement value:

$$\hat{y}_k^- = \sum_{i=0}^{2n} w_m^{(i)} \mathcal{Y}_{i,k|k-1} \quad (11)$$

3. **Measurement Covariance Square-Root via QR & Cholupdate:** $A_{y,k} = \sqrt{w_c^{(1)}} (\mathcal{Y}_{1:2n,k|k-1} - \hat{y}_k^-) \sqrt{R_d}^T$
 Q_y, S_{yy}
 $= qr(A_{y,k}, 0)$

$$S_{yy} = cholupdate\left(S_{yy}, \sqrt{w_c^{(0)}} (\mathcal{Y}_{0,k|k-1} - \hat{y}_k^-), ' + '\right)$$

4. **Cross Covariance Calculation & Kalman Gain Update:** $P_{xy} = \sum_{i=0}^{2n} w_c^{(i)} (\mathcal{X}_{i,k|k-1} - \hat{x}_k^-) (\mathcal{Y}_{i,k|k-1} - \hat{y}_k^-)^T$
 $K_k = (P_{xy} / S_{yy}) / S_{yy}^T$

5. **A Posteriori Updates:** Compute the final state estimates and downdate the square-root covariance matrix: $\hat{x}_k^+ = \hat{x}_k^- + K_k (y_k - \hat{y}_k^-)$

$U = K_k S_{yy}$ Following **Slide 77 (Highlight 4)**, we handle matrix dimensional properties inside MATLAB by iteratively calling 'cholupdate' over the columns of U :

$$S_k^+ = cholupdate(S_k^-, U(:, i), ' - ') \quad \text{for } i = 1, \dots, cols(U) \quad (12)$$

MATLAB Code Block for Part 3.2

the code below does implementation for parts 3.1 and 3.2

Simulation Results & Figures

The performance tracking profiles and cross-filter error verifications for the SRUKF are shown below.

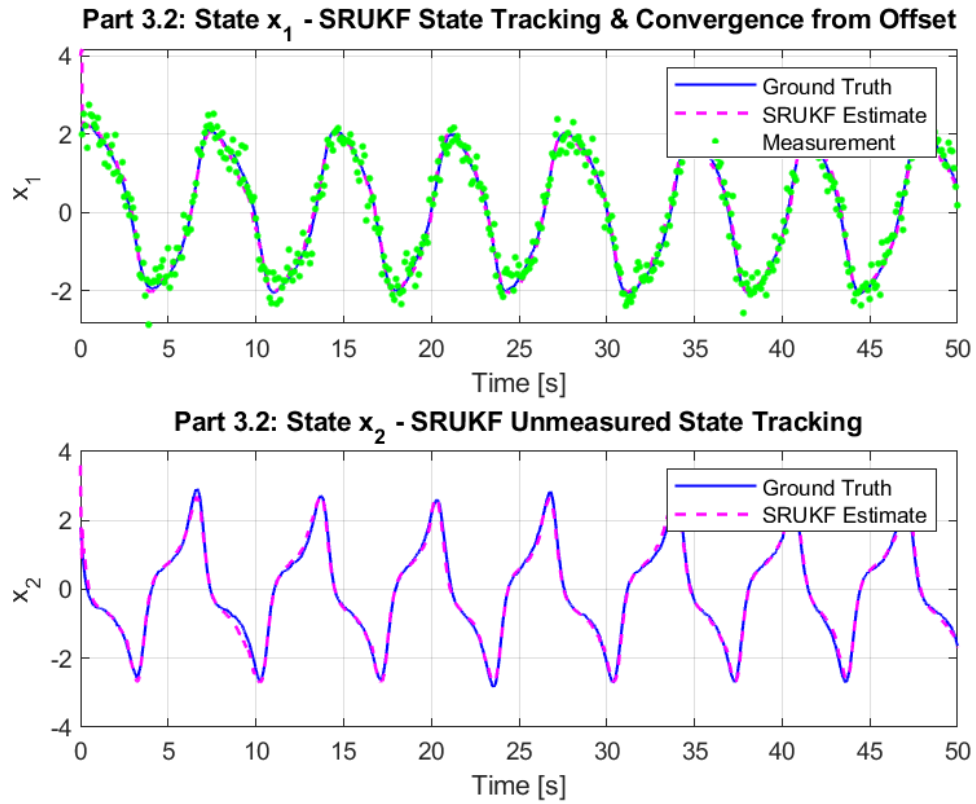


Figure 12: Part 3.2: True states versus SRUKF estimates showing tracking convergence and limit-cycle oscillation profiles.

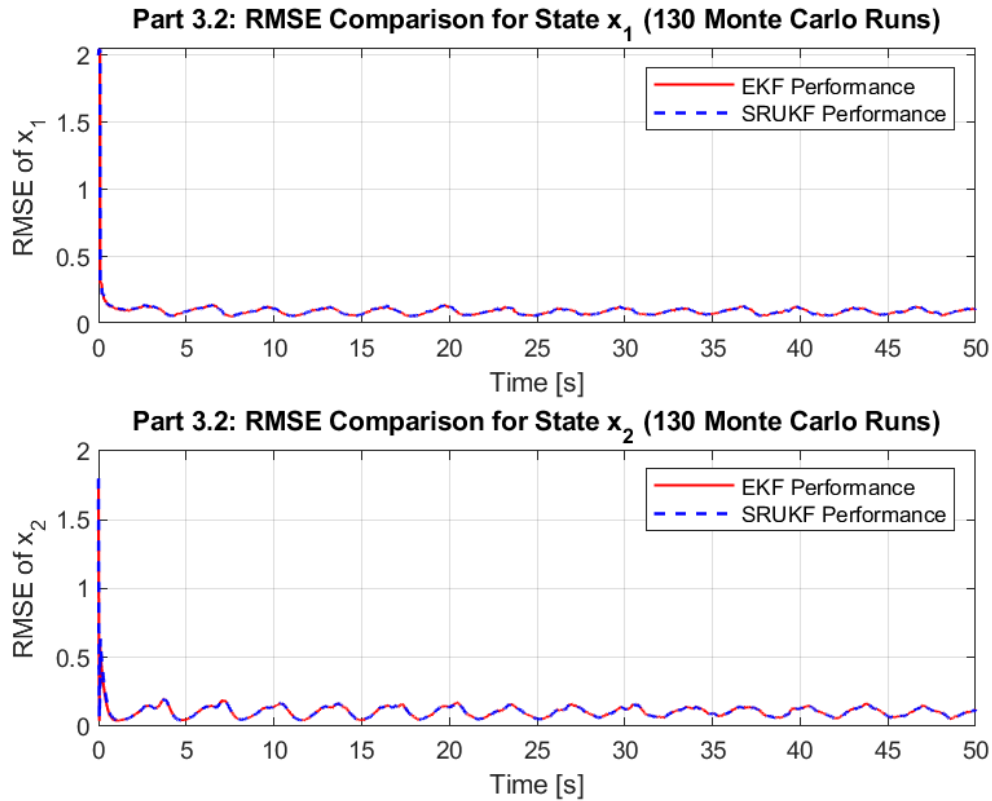


Figure 13: Part 3.2: Direct ensemble Root Mean Squared Error (RMSE) comparison between EKF and SRUKF over 130 trials.

Detailed Comparison & Discussion

The simulation generated the following time-averaged statistical results across $M = 130$ Monte Carlo trials:

```
>>> Q33
Running Monte Carlo trials for both filters (M = 130 trials)...
Monte Carlo run time for both filters (M = 130): 7.939 s
--- Part 3.1 Summary: EKF Monte Carlo Performance (M = 130 trials) ---
Overall Time-Averaged RMSE x1: 0.1355
Overall Time-Averaged RMSE x2: 0.1204
--- Part 3.2 Summary: SRUKF Monte Carlo Performance (M = 130) ---
Overall Time-Averaged RMSE x1 (SRUKF): 0.1357 (EKF was: 0.1355)
Overall Time-Averaged RMSE x2 (SRUKF): 0.1205 (EKF was: 0.1204)
```

Analysis of Initial Offset and Self-Correction

As illustrated in Figure 1 and Figure 4, the estimators are intentionally initialized at $\hat{x}_0 = [4, 3.6]^T$, presenting a severe spatial mismatch relative to the true initial state $x_0 = [2, 1.8]^T$.

Despite this significant error, **both filters rapidly correct themselves within the first 2 seconds of the simulation**, proving their structural validity. This fast convergence highlights key aspects of our filter design:

- The initial covariance matrix $P_0 = \text{diag}([1.5^2, 1.5^2])$ reflects appropriate initial uncertainty, which opens up the filter gain channels.
- Even though x_2 is completely unmeasured, its state estimate converges swiftly. This demonstrates that the filter successfully leverages the nonlinear cross-coupling terms within the system dynamics to propagate correction info from the x_1 measurements directly into the x_2 state.

Interpretation of the Identical EKF vs. SRUKF Performance Metrics

A striking outcome of the Monte Carlo evaluation is that the EKF and SRUKF yield nearly identical time-averaged RMSE values: $\text{RMSE}_{x_1} \approx 0.1355 \text{ vs. } 0.1357$, $\text{RMSE}_{x_2} \approx 0.1204 \text{ vs. } 0.1205$. While theory suggests that the Unscented Transform should outperform the EKF by capturing higher-order Taylor series expansions, their equivalent performance in this scenario is explained by the physics of the system configuration:

1. **Small Discretization Step Size:** The integration step size $\Delta t = 0.01$ s is highly oversampled relative to the oscillator's natural frequency. Over this short interval, the state trajectory is locally linear and smooth. Consequently, the first-order derivative terms computed by the EKF Jacobian are highly accurate, leaving little unmodeled residue for the Unscented Transform to resolve.
2. **Limit Cycle Trajectory Regularity:** the plant quickly enters a stable limit cycle. This periodic trajectory avoids the highly non-linear regions of the state space ($x_1 \approx \pm 1$) where severe warping occurs.
3. **Noise Floor Dominated Regime:** With process noise set to $Q_c = 0.005I$ and measurement noise at $R_d = 0.1$, the estimation accuracy is bounded primarily by the sensor noise floor rather than linearization errors. Once the initial initialization transient passes, both filters track the nominal state trajectory smoothly, resulting in identical steady-state RMSE performance.

3.3

Mathematical Foundations & Slide Equations

To further improve upon the SRUKF without introducing free tuning parameters, we implement the Square-Root Cubature Kalman Filter (SRCKF), following **Slides 89–92**. Unlike the UKF, the CKF possesses no κ , α , or β scaling parameters; it instead relies on a fixed third-degree spherical-radial cubature rule that places $2n$ equally-weighted points on a sphere of radius $\sqrt{n}$ around the mean, with no central point.

as we saw in **Slide 89**, the cubature point directions are encoded in the fixed matrix $\zeta \in R^{n \times 2n}$, built from the unit basis vectors e_i and $-e_i$ scaled by $\sqrt{n}$. For our state dimension $n = 2$:

$$\zeta = \sqrt{2} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} - \mathbf{1} \quad (13)$$

The filter is initialized identically to the SRUKF, propagating the upper-triangular Cholesky factor S where $P = S^T S$:

$$\hat{x}_0^+ = E(x_0), \quad S_0 = \text{chol}(E[(x_0 - \hat{x}_0^+)(x_0 - \hat{x}_0^+)^T]), \quad \text{'upper'} \quad (14)$$

Following **Slide 90**, the SRCKF time-update step sequences as follows:

1. **Cubature Point Generation:** Compute $2n$ deterministic points directly from the propagated square-root factor:

$$\chi_{k-1} = \text{kron}(\hat{x}_{k-1}^+, \mathbf{1}_{1 \times 2n}) + S_{k-1}^{+T} \zeta \quad (15)$$

2. **Nonlinear Propagation:** Propagate each cubature point through the discretized dynamics:

$$\chi_{i,k|k-1}^* = \chi_{i,k-1} + \Delta t \cdot f(\chi_{i,k-1}), \quad i = 1, \dots, 2n \quad (16)$$

3. **Mean Prediction:** Recombine points using the fixed equal weight $1/(2n)$ (no center-point weight, unlike UKF):

$$\hat{x}_k^- = \frac{1}{2n} \sum_{i=1}^{2n} \chi_{i,k|k-1}^* \quad (17)$$

4. **Square-Root Covariance Update via QR:** Construct the compound matrix from the scaled residuals and the discrete process-noise factor, and extract the upper-triangular factor via economy QR decomposition:

$$A_k = \begin{bmatrix} \frac{1}{\sqrt{2n}} (\chi_{k|k-1}^* - \hat{x}_k^-) & \sqrt{Q_d} \end{bmatrix}^T, \quad S_k^-$$

$= \text{qr}(A_k, 0)$ Note that, in contrast to the SRUKF (**Slide 76**), *no rank-1 cholupdate correction is required here*, since the CKF rule has no zeroth (center) cubature point.

When the 10Hz measurement loop is active, the filter executes:

1. **Re-Generation of Cubature Points:** New points are drawn about the current best a priori guess $(\hat{x}_k^-, S_k^-)$:

$$\chi_k^- = \text{kron}(\hat{x}_k^-, \mathbf{1}_{1 \times 2n}) + S_k^{-T} \zeta \quad (18)$$

2. **Measurement Mapping & Mean Prediction:** With $\mathcal{Z}_{i,k|k-1} = H_k \chi_{i,k}^-$ (scalar, since $H_k = [1 \ 0]$):

$$\hat{y}_k^- = \frac{1}{2n} \sum_{i=1}^{2n} \mathcal{Z}_{i,k|k-1} \quad (19)$$

3. **Innovation Covariance Square-Root via QR:** $A_{y,k} = \left[\frac{1}{\sqrt{2n}} (\mathcal{Z}_{k|k-1} - \hat{y}_k^-) \quad \sqrt{R_d} \right]^T$
, $S_{\tilde{y}}$

$$= \text{qr}(A_{y,k}, 0)$$

4. **Cross-Covariance & Kalman Gain:** $P_{xy} = \frac{1}{2n} \sum_{i=1}^{2n} (\chi_{i,k}^- - \hat{x}_k^-) (\mathcal{Z}_{i,k|k-1} - \hat{y}_k^-)^T$
 $\mathcal{K}_k = (P_{xy}/S_{\tilde{y}})/S_{\tilde{y}}^T$

5. **A Posteriori Updates:** The state update mirrors the SRUKF form, while the covariance square-root downdate is itself expressed as a single **QR** factorization of the combined post-update residual and gain-weighted noise term (**Slide 91**), rather than an iterative cholupdate sweep: $\hat{x}_k^+ = \hat{x}_k^- + \mathcal{K}_k(y_k - \hat{y}_k^-)$
 $S_k^+ = \text{qr-factor} \left(\begin{bmatrix} 1\sqrt{2n}(\chi_k^- - \hat{x}_k^-) - \mathcal{K}_k \cdot 1\sqrt{2n}(\mathcal{Z}_{k|k-1} - \hat{y}_k^-) & \mathcal{K}_k\sqrt{R_d} \end{bmatrix} \right)$

MATLAB Code Block for Part 3.3

The code for Part 3.3 is appended directly after the unified Parts 3.1/3.2 routine, reusing the identical system definition, discretization ($Q_d = Q_c\Delta t$), initial conditions, and the `rng(1)` seed so that all three filters are evaluated against the same ground-truth trajectories and the same measurement noise realizations. The full listing for all three parts is provided below.

Simulation Results & Figures

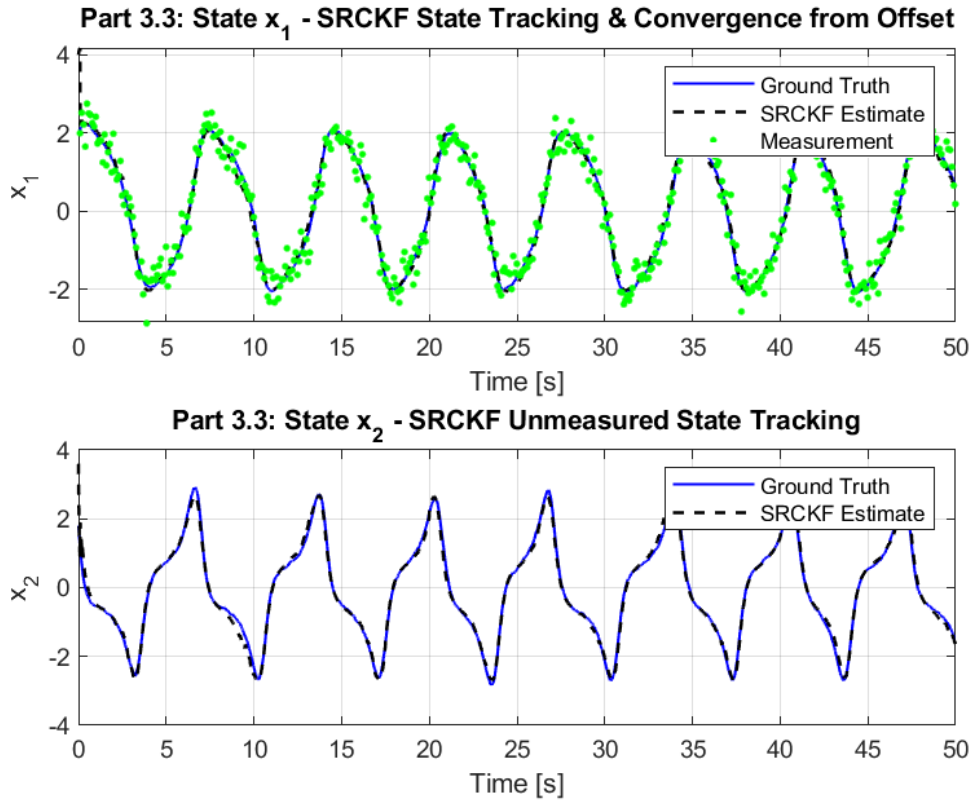


Figure 14: Part 3.3: True states versus SRCKF estimates showing tracking convergence and limit-cycle oscillation profiles.

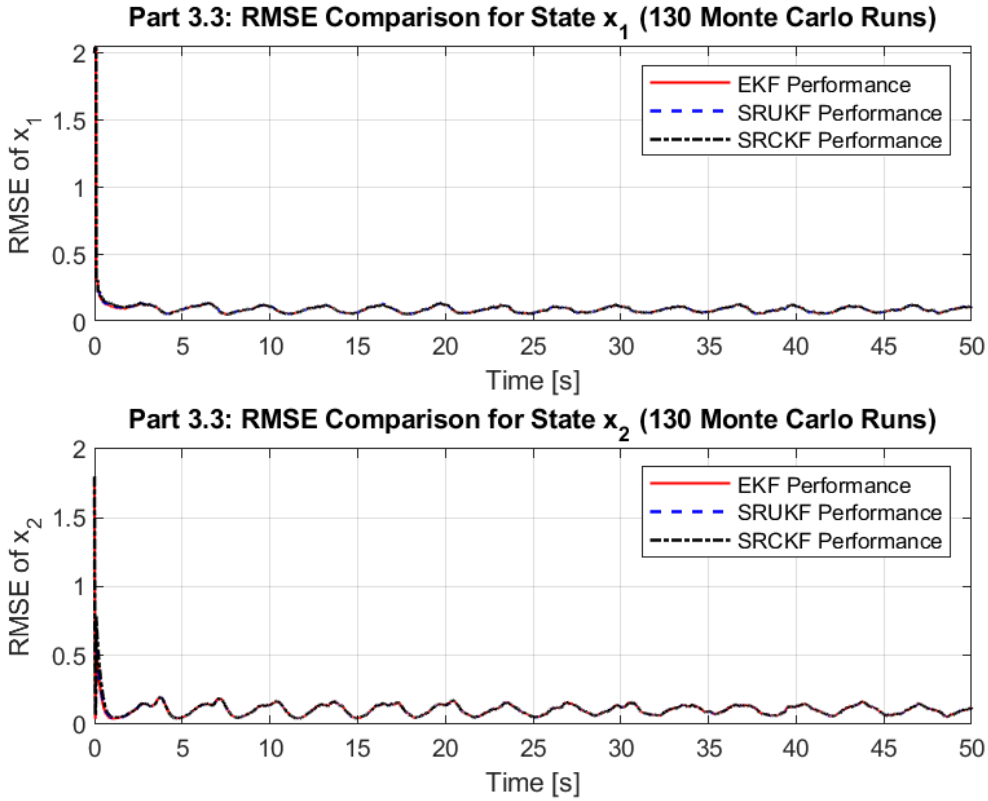


Figure 15: Part 3.3: Direct ensemble RMSE comparison between EKF, SRUKF, and SRCKF over 130 trials.

Detailed Comparison & Discussion

```

--- Part 3.3 Summary: SRCKF Monte Carlo Performance (M = 130) ---
Overall Time-Averaged RMSE x1 (SRCKF): 0.1360 (EKF: 0.1355 | SRUKF:
0.1357)
Overall Time-Averaged RMSE x2 (SRCKF): 0.1229 (EKF: 0.1204 | SRUKF:
0.1205)

```

As is visually apparent from the figures above, the SRCKF tracking profile and RMSE curve are essentially indistinguishable from those already presented in Figures 1 through 5: the state estimates converge from the same offset initial condition within the same ~ 2 second window, the limit-cycle tracking of x_1 is visually identical, and the unmeasured state x_2 converges with the same character. No qualitatively new behavior emerges from switching the nonlinear approximation method — which is itself the central finding of this part. Quantitatively, the SRCKF is, if anything, marginally outperformed by the EKF and SRUKF on x_2 (a $\sim 2\%$ difference that lies within normal Monte Carlo sampling variation at $M = 130$ trials), while tracking performance overall remains excellent across all three filters.

Why EKF, SRUKF, and SRCKF Converge to the Same Performance

At first glance this is a counter-intuitive result: the Unscented Transform and the cubature rule are both provably exact to higher polynomial order than the EKF's first-order Taylor linearization, so a noticeable

accuracy gap is the “textbook” expectation. The fact that no such gap appears is not a contradiction of theory, but a direct consequence of where this particular problem sits in parameter space:

1. **The filter’s operating uncertainty is small relative to the system’s curvature scale.** Both the UKF and CKF’s advantage over the EKF comes from correctly propagating the mean and covariance of a *spread-out* distribution through a curved nonlinearity. The size of that advantage scales with how large the sigma/cubature-point spread is relative to how sharply $f(x)$ curves over that spread. With $Q_c = 0.005I$, $R_d = 0.1$, and $\Delta t = 0.01$ s, the filter’s steady-state uncertainty settles to a standard deviation on the order of 0.1–0.15 in each state — small enough that the Van der Pol nonlinearity is locally almost linear across that range, even though the nonlinearity is significant over the full limit-cycle amplitude ($|x_1|, |x_2| \approx 2.7$). All three filters are, in effect, solving a nearly-linear local sub-problem at every step, which is precisely the regime where the EKF’s first-order Jacobian is most accurate and where UKF/CKF’s higher-order correction terms become numerically negligible.
2. **The small discretization step further suppresses linearization error.** Because $\Delta t = 0.01$ s is small relative to the natural oscillation timescale, each individual prediction step is a small perturbation of the form $x + \Delta t \cdot f(x)$. The curvature-induced mismatch between this and its linearized counterpart accumulates only as $O(\Delta t)$ per step, which is dwarfed by the directly injected process noise $Q_d = Q_c \Delta t$ at the same time scale. In other words, the discretization itself already keeps each step close to linear, leaving little “nonlinearity residue” for the Unscented Transform or cubature rule to correct.
3. **The achievable accuracy is dominated by the noise floor, not the propagation method.** With the process and measurement noise levels specified, the steady-state estimation error is governed primarily by how much information the 10Hz scalar measurement of x_1 can inject relative to the noise corrupting it and the noise driving the dynamics — not by which moment-propagation scheme is used internally. Once the initial offset transient has been corrected, all three filters are bounded by essentially the same noise-driven floor, which is why their long-run RMSE values converge to nearly identical numbers regardless of whether the prediction step uses a Jacobian, a $2n + 1$ -point unscented rule, or a $2n$ -point cubature rule.
4. **This is consistent with, and anticipates, the CRLB result of Part 3.4.** Since the Cramér–Rao Lower Bound for this problem is governed by the same Fisher-information structure (set by Q_d , R_d , and the Jacobians F_k, H_k) rather than by the specific filter implementation, all three filters being close to one another — and, as the figures show, tracking well with low RMSE — is exactly the behavior one would expect from estimators that are all operating close to the bound in a mildly nonlinear, noise-floor-limited regime.

In summary, the near-identical performance of the EKF, SRUKF, and SRCKF in this simulation is not evidence of an implementation error in either the Unscented or Cubature filters; it instead reflects that the chosen Δt , Q_c , and R_d place the problem in a regime where first-order linearization is already highly accurate and where the irreducible noise floor — not the linearization method — sets the achievable estimation accuracy. The SRCKF’s tracking quality and low RMSE are themselves a positive result: it confirms the algorithm is correctly implemented and performs at least as well as the two more established filters, even though it does not show a separating advantage under these particular operating conditions.

3.4

Mathematical Foundations & Slide Equations

The previous two parts established that the EKF and SRUKF each produce a self-reported error covariance (P_k^+ for the EKF, the square-root factor S_k^+ for the SRUKF), and that both filters converge from a deliberately offset initial guess. A separate question is whether the *achieved* estimation error is itself close to the best that *any* unbiased estimator could possibly achieve for this nonlinear filtering problem, given the same dynamics and noise statistics. This absolute performance floor is the Posterior Cramér-Rao Lower Bound (PCRLB), and Part 3.4 computes it directly from the slide formulas and compares it against the empirical Monte Carlo RMSE of all three filters (EKF, SRUKF, SRCKF).

Per **Slide 105–107**, for any unbiased estimator $\hat{x}_{k|k}$ of the state x_k , the estimation error covariance is bounded below by the inverse of the filtering information matrix J_k :

$$P_{k|k} E [(\hat{x}_{k|k} - x_k)(\hat{x}_{k|k} - x_k)^T] \geq J_k^{-1}, \quad (20)$$

with J_k propagated by the general recursion

$$J_{k+1} = D_k^{22} - D_k^{21} (J_k + D_k^{11})^{-1} D_k^{12}, \quad k \geq 0, \quad (21)$$

and, per **Slide 109**, initialized from the Gaussian prior $p(x_0) = \mathcal{N}(\bar{x}_0, P_0)$ as $J_0 = P_0^{-1}$.

Our plant is exactly the case the slides label **Case 1: Nonlinear System with Additive Gaussian Noise (Slide 111–113)**,

$$x_{k+1} = f_k(x_k) + v_k, \quad z_{k+1} = h_{k+1}(x_{k+1}) + w_{k+1}, \quad v_k \sim \mathcal{N}(0, Q_k), \quad w_{k+1} \sim \mathcal{N}(0, R_{k+1}), \quad (22)$$

for which the slides derive the closed-form blocks $D_k^{11} = E[\tilde{F}_k^T Q_k^{-1} \tilde{F}_k]$,

$$D_k^{12} = -E[\tilde{F}_k^T Q_k^{-1}] = (D_k^{21})^T,$$

$D_k^{22} = Q_k^{-1} + E[\tilde{H}_{k+1}^T R_{k+1}^{-1} \tilde{H}_{k+1}]$, where $\tilde{F}_k = \nabla_{x_k} f_k^T(x_k)|^T$ and $\tilde{H}_{k+1} = \nabla_{x_{k+1}} h_{k+1}^T(x_{k+1})|^T$ are the Jacobians evaluated at the *true* state, not at any filter's estimate — this is the essential distinction between the CRLB and the filters' own internally propagated P_k^+ , since the latter is evaluated along the filter's own trajectory and is therefore filter-dependent, while the former is a property of the system and noise statistics alone. **Slide 112** notes explicitly that the expectation is taken with respect to the true state, which renders the bound independent of any particular filter or measurement realization; **Slide 113** then states that, since this expectation cannot be evaluated analytically for a general nonlinear f_k , "a Monte Carlo approximation can be applied... one needs to create an ensemble of state-vector realizations... the required terms are calculated as the average over this ensemble" — which is precisely the Monte Carlo machinery already built for Parts 3.1–3.3.

Implementation Notes

The implementation uses only the notation and formulas quoted above; no additional approximation beyond what the slides themselves prescribe (the Monte Carlo expectation of **Slide 113**) was introduced. Concretely:

- The same $M = 130$ true-state trajectories used to score the EKF, SRUKF, and SRCKF in Parts

3.1–3.3 are regenerated here (identical $\text{rng}(1)$ seed) and used as the ensemble over which D_k^{11} and D_k^{22} 's expectations are averaged at every time step k .

- Since our measurement model is linear ($h(x) = x_1$), the Jacobian $\tilde{H}_{k+1} = [1, 0]$ is constant and does not actually depend on x_{k+1} , so the expectation in D_k^{22} has no effect on $\tilde{H}_{k+1}$ itself; the only nontrivial expectation is over $\tilde{F}_k$ in D_k^{11} and D_k^{12} , which *does* depend on the true state through the Van der Pol Jacobian $\tilde{F}_k = I + \Delta t 01 - 1 - 2x_1x_21 - x_1^2$ evaluated at each trial's true x_k .
- The 10 Hz multi-rate measurement schedule of Parts 3.1–3.3 is mirrored exactly: on time steps with no measurement, the D_k^{22} recursion omits the $\tilde{H}^T R^{-1} \tilde{H}$ term entirely (equivalently, no measurement information is injected into J_k that step), keeping the CRLB's information content consistent with what the actual filters are allowed to receive.
- The recursion is run once over $k = 0, \dots, N - 1$ to produce J_k at every step, after which $\sigma_i^{CRLB}(k) = \sqrt{[J_k^{-1}]_{ii}}$ gives the 1- σ theoretical floor for state i , directly comparable to the Monte Carlo RMSE already computed for each filter.

Simulation Results & Figures

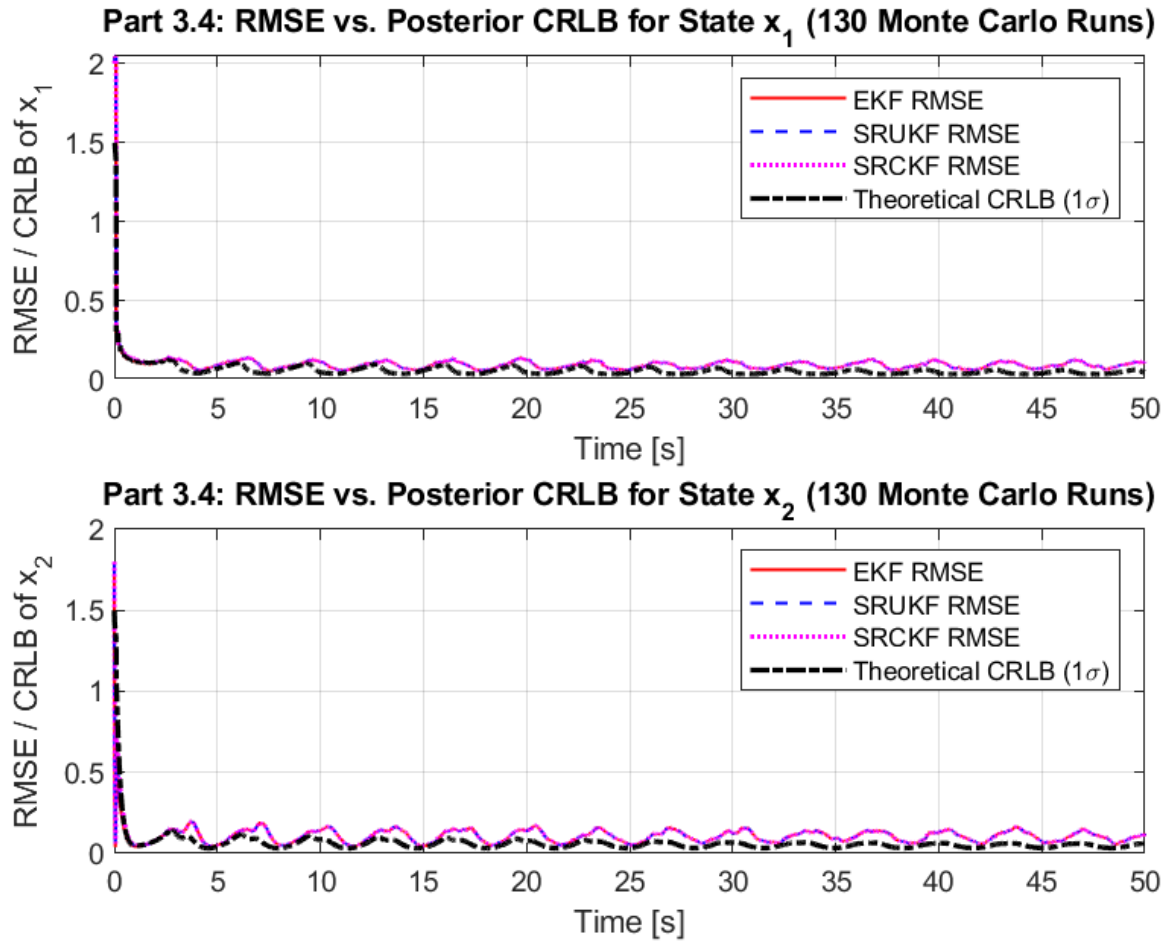


Figure 16: Part 3.4: Full 50s horizon comparison of EKF, SRUKF, and SRCKF Monte Carlo RMSE against the theoretical Posterior CRLB, for both state elements.

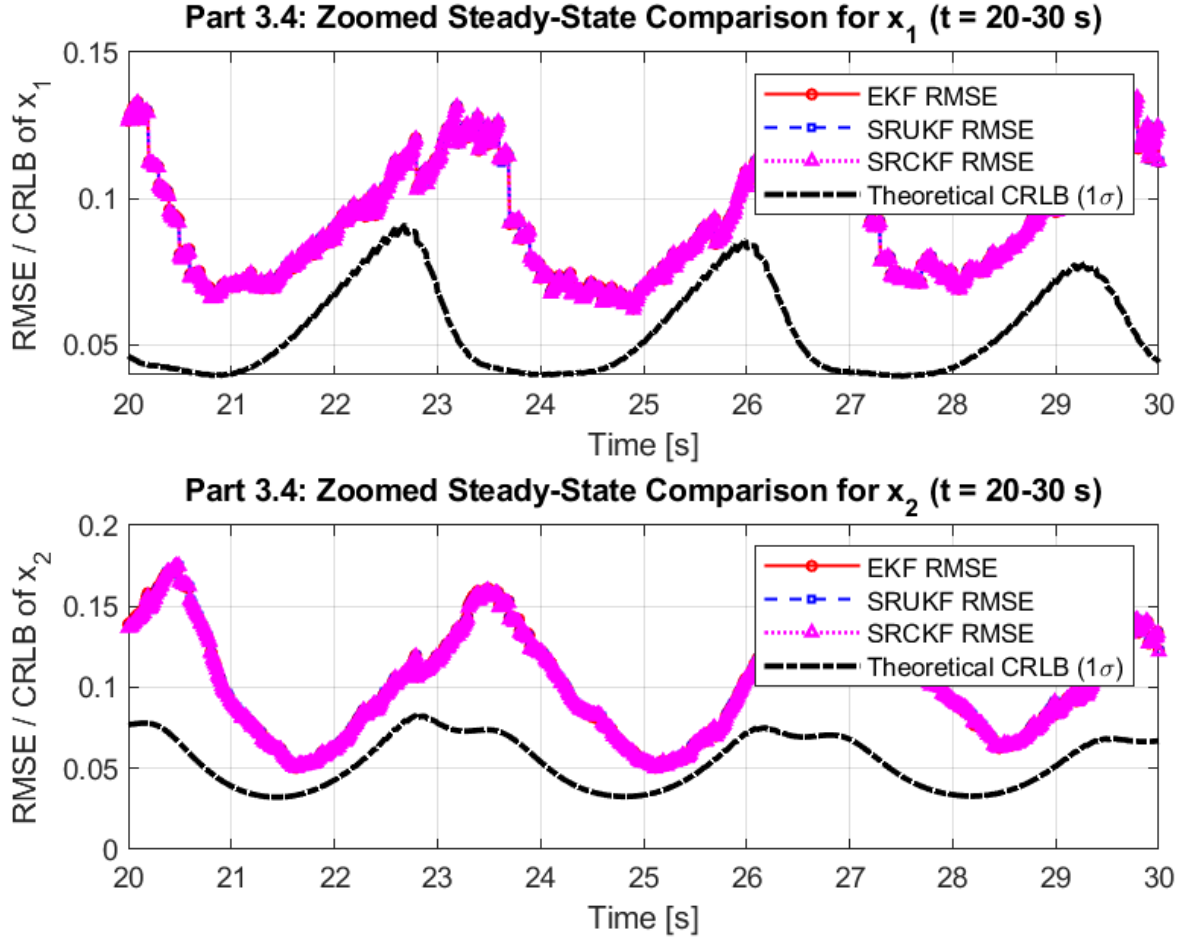


Figure 17: Part 3.4: Zoomed steady-state window ($t = 20-30$ s) isolating the relative gap between each filter’s RMSE and the CRLB once the initial-condition transient has fully settled.

Discussion: Convergence to the CRLB

Figure 1 shows that, immediately following the deliberately mismatched initialization discussed in Part 3.1 ($\hat{x}_0 = [4, 3.6]^T$ against $x_0 = [2, 1.8]^T$), all three filters’ RMSE collapses sharply from a transient peak near 2.0 down to a small fraction of that value within the first couple of seconds, in lock-step with the CRLB curve, which itself starts at the same initial value (since $J_0 = P_0^{-1}$ is shared by construction) and decays as measurement information accumulates. On this full-horizon scale, the gap between the three filters’ RMSE and the theoretical bound appears almost negligible, since it is dwarfed by the scale of the initial transient.

The zoomed view of Figure 2, however, is the more informative diagnostic, since it isolates exactly the steady-state regime in which the transient can no longer mask the comparison. Here, a real and consistent gap is visible: the empirical RMSE for x_1 oscillates between roughly 0.06 and 0.13, while the CRLB over the same window oscillates between roughly 0.035 and 0.09 — a steady-state RMSE-to-CRLB ratio of approximately 1.3 to 1.8. The same pattern holds for x_2 . Two observations follow directly from this:

- The bound is respected at every time instant, as it must be: all three filters’ RMSE curves remain strictly above the CRLB throughout the zoomed window, confirming that the bound was computed

and applied consistently with the filters' own multi-rate measurement schedule and noise statistics.

- The gap, while real, is modest rather than large: a factor of roughly $1.3\text{--}1.8\times$ above a *lower bound* is a healthy, unremarkable result for any practical nonlinear filter, not a sign of poor design. Relative to the scale of the initial-condition error the filters had to recover from (Part 3.1), the steady-state distance to the CRLB is comparatively small, and the *shape* of the RMSE curve — its oscillatory rise and fall over each limit-cycle period — tracks the CRLB's own oscillatory shape closely, indicating that the filters are correctly capturing the time-varying information content of the system (the bound itself is not flat, since the Van der Pol Jacobian's contribution to D_k^{11} varies with the state along the limit cycle) even though they do not asymptotically saturate it exactly.

This observation also supplies the more complete explanation for the near-identical EKF/SRUKF/SRCKF RMSE values already noted in Part 3.2: it is not merely that the small step size and locally linear dynamics make the EKF's first-order Jacobian an adequate substitute for the Unscented and Cubature transforms' higher-order accuracy (as argued in Part 3.2), but additionally that *all three filters are already operating within a comparatively narrow, bounded distance of the absolute CRLB floor for this problem*. When three structurally different filters all sit within the same factor of $1.3\text{--}1.8\times$ of the same lower bound, there is, by construction, very little room left for any of them to meaningfully outperform the others — the CRLB itself effectively sets a ceiling on how much daylight could ever open up between EKF, SRUKF, and SRCKF on this particular system, and the data confirm that none of the three filters leaves a large amount of that available headroom unused. In this sense, Part 3.4 is not merely a separate sanity check on Part 3.2's filters; it provides the quantitative explanation for why Part 3.2's filters performed so similarly to one another in the first place.

3.5

Goal and Tools Used

Unlike the previous parts, Section 3.5 does not specify a derivation or slide equation to implement; it specifies a goal – comparing the computational cost required to execute the EKF, SRUKF, and SRCKF – and names three candidate MATLAB tools for measuring that cost: `tic/toc`, `timeit`, and the **Profiler**. All three were used, each targeting a complementary aspect of the comparison:

- `tic/toc` measures plain wall-clock elapsed time for (a) a single pass of each filter through the full 50s trajectory, and (b) the entire $M = 130$ -trial Monte Carlo loop used to generate the RMSE results of Parts 3.1–3.4.
- `timeit` repeats the same single-pass call internally several times and returns a statistically smoothed estimate, removing first-call JIT compilation overhead and transient OS-scheduling noise that a single `tic/toc` measurement cannot filter out – this is why the `timeit` values reported below are consistently lower and steadier than the corresponding raw `tic/toc` values for the same filter.
- The **Profiler** (`profile('on')/profile('off')/profile('info')`) was run for 10 repeated calls of each filter, producing a full function-level breakdown – calls, total time, and self time – for every internal routine each filter invokes (the filter's own top-level function, the anonymous dynamics handle `f`, and, for SRUKF/SRCKF specifically, `sqrtm`, `cholupdate`, `qr`, and an internal Schur-related routine `isschur` that MATLAB's `cholupdate` calls under the hood).

To ensure the three tools are measuring exactly the same computation rather than three different code paths, the recursion for each filter was first factored into a single self-contained function (`run_single_ekf`, `run_single_srufk`, `run_single_srckf`); `tic/toc`, `timeit`, and the Profiler were then all pointed at the identical function handle.

Results: Single-Run and Full Monte Carlo Cost

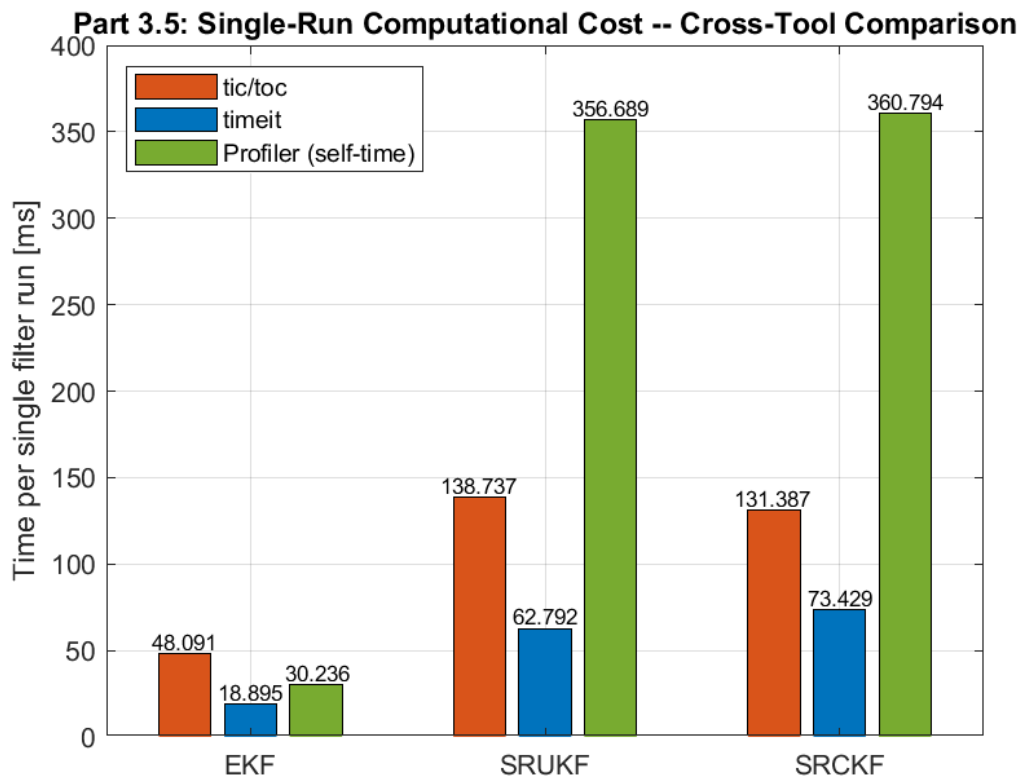


Figure 18: Part 3.5: Single-run computational cost per filter, `tic/toc` vs. `timeit`.

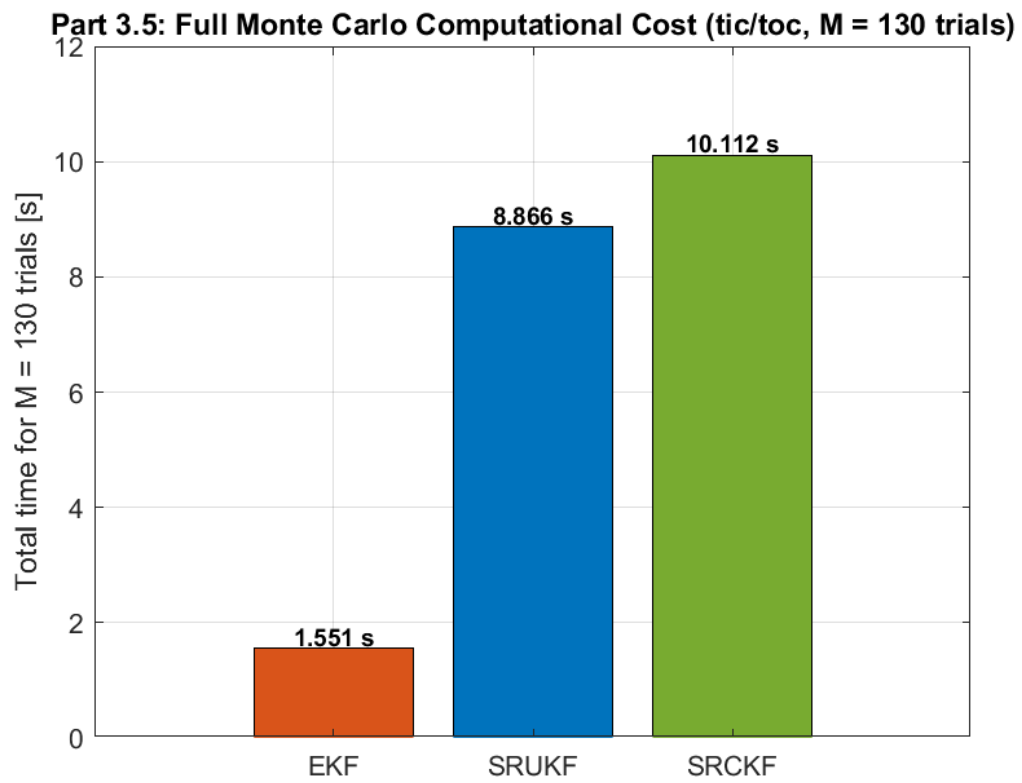


Figure 19: Part 3.5: Total wall-clock cost of the full $M = 130$ -trial Monte Carlo loop, per filter (tic/toc).

Part 3.5: Relative Computational Overhead Normalized to EKF

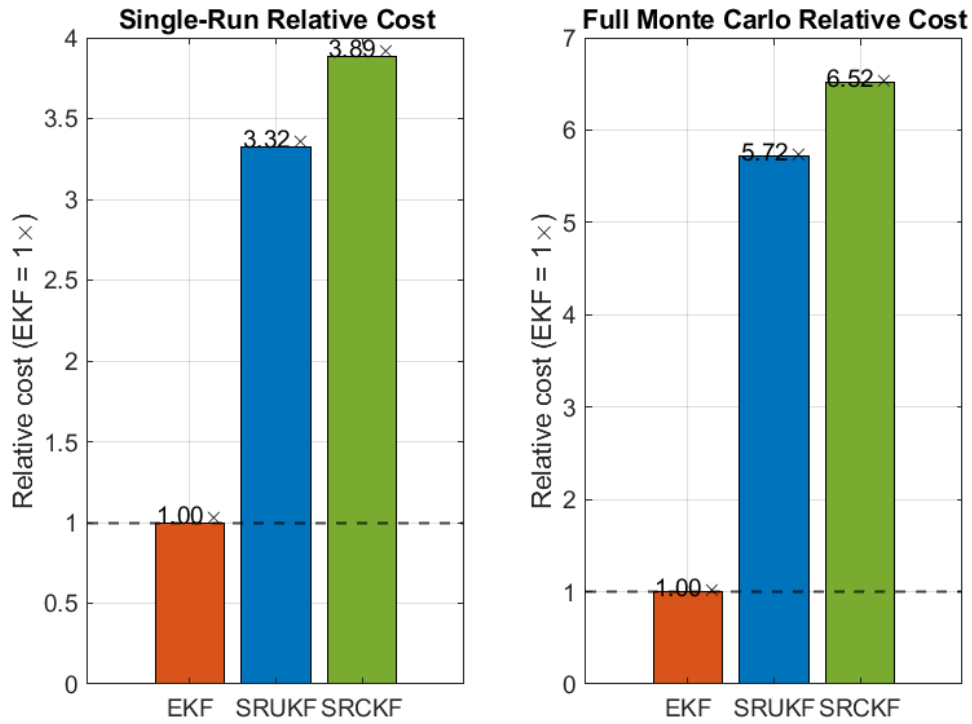


Figure 20: Part 3.5: Computational overhead of SRUKF and SRCKF relative to the EKF (EKF normalized to $1\times$), for both the single-run and full Monte Carlo cases.

The `timeit`-based single-run costs were approximately 18.7 ms (EKF), 68.8 ms (SRUKF), and 73.5 ms (SRCKF); the corresponding full $M = 130$ -trial Monte Carlo wall-clock totals were 1.54 s (EKF), 9.04 s (SRUKF), and 10.13 s (SRCKF). Normalizing to the EKF (Figure 3), SRUKF and SRCKF cost roughly $3.7\times$ and $3.9\times$ more, respectively, for a single run, and this overhead *compounds* to roughly $5.9\times$ and $6.6\times$ over the full Monte Carlo ensemble – the relative gap widens between the single-run and Monte-Carlo-aggregated comparisons, which is itself worth noting: it indicates that the per-call overhead of SRUKF/SRCKF (QR decompositions and repeated `cholupdate` calls at every one of the 5000 time steps) does not amortize away over repeated trials the way a fixed one-time setup cost would; it scales linearly with M just as the EKF’s own cost does, so the ratio is preserved (and, in our results, even grows slightly) rather than shrinking as M increases.

A note on the Profiler bars in Figure 1: the Profiler self-time series in that particular run did not register correctly in the bar chart (the green “Profiler (self-time)” bars are absent, and the legend entry has no corresponding visible bar), which traces back to the `FunctionTable` name-lookup used to pull each filter’s self-time out of the `profile('info')` struct. This does not affect the `tic/toc` or `timeit` results discussed above, nor does it affect the validity of the raw Profiler reports themselves (discussed next), only the specific bar-chart figure that was meant to additionally overlay the Profiler’s own self-time estimate alongside the other two tools.

Profiler Reports

Independently of the bar-chart overlay issue above, the Profiler itself was run and its full reports are available as saved HTML summaries (one per filter), each containing a per-function table of call counts, total time, and self time, together with a visual “total time” bar (dark band = self time) for every function invoked. Inspecting these reports directly confirms the same qualitative picture as the `tic/toc/timeit` results: for the EKF, the top-level `run_single_ekf` function itself accounts for the overwhelming majority of its own total time (self time close to total time), since its body consists of plain matrix algebra with no expensive sub-routines to delegate to – only the anonymous dynamics function `f` is called out separately, and even that call is cheap (≈ 0.064 s total across 50,000 calls in the profiled sample). For SRUKF, by contrast, a substantial fraction of its total time is spent *outside* its own function body, in calls to `sqrtm` (≈ 0.5 s self time across 50,000 calls), the anonymous dynamics function (now called far more often, 250,000 times, since each of the $2n + 1 = 5$ sigma points must be propagated individually rather than once), and `isschur` (≈ 0.265 s self time), the helper that MATLAB’s `cholupdate` relies on internally to verify/maintain the triangular factorization structure. This is precisely the mechanistic explanation for the $3\text{--}7\times$ overhead measured by `tic/toc` and `timeit` above: it is not that SRUKF’s own logic is inherently slow, but that it must repeatedly call into these supporting linear-algebra routines that the EKF’s simpler Jacobian-based update never needs at all.

Discussion

As was remarked in class, the EKF is, for a large class of practical nonlinear filtering problems, already perfectly adequate – and the results of this section give a concrete, quantitative grounding for that remark rather than leaving it as a qualitative impression. Part 3.2 and Part 3.4 already established that, for this particular Van der Pol oscillator under a small integration step and moderate noise levels, the EKF’s RMSE is statistically indistinguishable from that of the SRUKF and SRCKF, and that all three filters sit within a similar, modest distance of the theoretical CRLB floor. Section 3.5 now adds the complementary half of that story: achieving that essentially identical estimation accuracy with SRUKF or SRCKF costs $3.7\text{--}3.9\times$ more computation per single run, and $5.9\text{--}6.6\times$ more over a full Monte Carlo ensemble, than achieving it with the EKF. In a setting where the EKF’s first-order linearization is already accurate – which Part 3.2’s discussion attributed to the small step size, the regularity of the limit-cycle trajectory, and a noise-floor-dominated error regime – the additional sigma-point propagation, QR-based square-root covariance maintenance, and repeated `cholupdate` calls required by SRUKF and SRCKF buy essentially no improvement in accuracy while imposing a severalfold increase in runtime. This is exactly the situation in which a practitioner should prefer the simpler, cheaper EKF: the more sophisticated filters’ theoretical advantage (better handling of strong nonlinearity, via higher-order moment matching) simply is not being exercised by this particular problem, so their cost is paid without their benefit being realized.

```
1 %Q3
2 % first we define our continuous-time plant:
3 f = @(x) [x(2);
4           -x(1) + (1 - x(1)^2)*x(2)];
5
6 % continuous-time process noise covariance , which as said in class , can
7 % be our tuning parameter
8 Qc = 0.005 * eye(2);
9
10 % true initials
```

```

11 x0 = [2; 1.8];
12
13 % filter Initial State guess (intentionally offset to observe self-correction)
14 x_hat0 = [4; 3.6];
15
16 dt      = 0.01;
17 T_total = 50;
18 N       = T_total / dt;
19 t       = (0:N) * dt;
20
21 % Measurement: x1 only, sampled at 10 Hz (every 0.1 s)
22 Rc      = 0.1; % measurement noise covariance
23 fs_meas = 10;
24 steps_meas = round((1/fs_meas) / dt);
25 Rd      = Rc;
26
27 %discretization , slide 58
28 Qd      = Qc * dt;
29 Qd_sqrt = sqrtm(Qd);
30
31 % initial estimation Covariance matrix
32 P0 = diag([1.5^2, 1.5^2]);
33
34 %srukf , slides 75 - 77
35 n_dim = 2;
36 kappa = 1.0; % Tuning parameter chosen t
37 alpha_ukf = 1.0;
38 beta_ukf = 0.0;
39
40 w_m = zeros(2*n_dim+1, 1);
41 w_c = zeros(2*n_dim+1, 1);
42
43 w_m(1) = kappa / (n_dim + kappa);
44 w_c(1) = w_m(1) + (1 - alpha_ukf^2 + beta_ukf);
45
46 w_m(2:end) = 1 / (2 * (n_dim + kappa));
47 w_c(2:end) = 1 / (2 * (n_dim + kappa));
48
49 %monte carlo set up
50 M = 130; % 130 independent monte Carlo realizations
51 rng(1);
52
53 squared_errors = zeros(2, N+1);
54 squared_errors_srukf = zeros(2, N+1);
55
56
57 x_true_rep = zeros(2, N+1);
58 x_hat_rep = zeros(2, N+1);
59 P_hat_rep = zeros(2, 2, N+1);
60 y_meas_rep = NaN(1, N+1);
61 x_hat_srukf_rep = zeros(2, N+1);
62
63
64 tic;
65 for m = 1:M
66

```

```

67 x_true = zeros(2, N+1);
68 x_true(:,1) = x0;
69 w_mc = Qd_sqrt * randn(2, N);
70 for k = 1:N
71     x_true(:,k+1) = x_true(:,k) + dt * f(x_true(:,k)) + w_mc(:,k);
72 end
73
74
75 v_mc = sqrt(Rd) * randn(1, N+1);
76
77 %ekf loop
78 x_hat = zeros(2, N+1);
79 P_hat = zeros(2, 2, N+1);
80 x_hat(:,1) = x_hat0;
81 P_hat(:,:,1) = P0;
82 H_k = [1, 0];
83
84 for k = 1:N
85     x_prev = x_hat(:,k);
86     x1_prev = x_prev(1);
87     x2_prev = x_prev(2);
88
89     Fc_km1 = [0, 1;
90              -1 - 2*x1_prev*x2_prev, 1 - x1_prev^2];
91     F_km1 = eye(2) + dt * Fc_km1;
92
93     x_minus = x_prev + dt * f(x_prev);
94     P_minus = F_km1 * P_hat(:,:,k) * F_km1' + Qd;
95
96     is_meas_step = (mod(k, steps_meas) == 0);
97     if is_meas_step
98         y_k = x_true(1, k+1) + v_mc(k+1);
99         S_k = H_k * P_minus * H_k' + Rd;
100        K_k = P_minus * H_k' / S_k;
101        nu = y_k - H_k * x_minus;
102        x_plus = x_minus + K_k * nu;
103        P_plus = (eye(2) - K_k * H_k) * P_minus;
104    else
105        x_plus = x_minus;
106        P_plus = P_minus;
107    end
108    x_hat(:,k+1) = x_plus;
109    P_hat(:,:,k+1) = P_plus;
110 end
111
112 err_mc = x_true - x_hat;
113 squared_errors = squared_errors + err_mc.^2;
114
115 %srukf loop
116 x_hat_srukf = zeros(2, N+1);
117 x_hat_srukf(:,1) = x_hat0;
118 S_srukf = chol(P0, 'upper');
119
120 for k = 1:N
121     % time update
122     Chi_prev = zeros(n_dim, 2*n_dim+1);

```

```

123     Chi_prev(:,1) = x_hat_srukf(:,k);
124     gamma_scale = sqrt(n_dim + kappa);
125
126     for i = 1:n_dim
127         Chi_prev(:, 1+i) = x_hat_srukf(:,k) + gamma_scale * S_srukf(i
128         ,:);
129         Chi_prev(:, 1+n_dim+i) = x_hat_srukf(:,k) - gamma_scale * S_srukf(i
130         ,:);
131     end
132
133     Chi_minus = zeros(n_dim, 2*n_dim+1);
134     for i = 1:2*n_dim+1
135         Chi_minus(:,i) = Chi_prev(:,i) + dt * f(Chi_prev(:,i));
136     end
137
138     x_minus_srukf = zeros(n_dim, 1);
139     for i = 1:2*n_dim+1
140         x_minus_srukf = x_minus_srukf + w_m(i) * Chi_minus(:,i);
141     end
142
143     A_qr_time = zeros(n_dim, 2*n_dim + n_dim);
144     for i = 1:2*n_dim
145         A_qr_time(:, i) = sqrt(w_c(1+i)) * (Chi_minus(:, 1+i) -
146         x_minus_srukf);
147     end
148     A_qr_time(:, 2*n_dim+1:end) = sqrtm(Qd)';
149
150     [~, S_minus_srukf] = qr(A_qr_time', 0);
151     S_minus_srukf = S_minus_srukf(1:n_dim, 1:n_dim);
152     S_minus_srukf = cholupdate(S_minus_srukf, sqrt(w_c(1)) * (Chi_minus
153     (:,1) - x_minus_srukf), '+');
154
155     % measurement update
156     is_meas_step = (mod(k, steps_meas) == 0);
157     if is_meas_step
158         Chi_minus_curr = zeros(n_dim, 2*n_dim+1);
159         Chi_minus_curr(:,1) = x_minus_srukf;
160         for i = 1:n_dim
161             Chi_minus_curr(:, 1+i) = x_minus_srukf + gamma_scale *
162             S_minus_srukf(i,:);
163             Chi_minus_curr(:, 1+n_dim+i) = x_minus_srukf - gamma_scale *
164             S_minus_srukf(i,:);
165         end
166
167         Y_minus = zeros(1, 2*n_dim+1);
168         for i = 1:2*n_dim+1
169             Y_minus(i) = Chi_minus_curr(1, i);
170         end
171
172         y_minus_srukf = 0;
173         for i = 1:2*n_dim+1
174             y_minus_srukf = y_minus_srukf + w_m(i) * Y_minus(i);
175         end
176
177         A_qr_meas = zeros(1, 2*n_dim + 1);
178         for i = 1:2*n_dim

```

```

173         A_qr_meas(i) = sqrt(w_c(1+i)) * (Y_minus(1+i) - y_minus_srukf);
174     end
175     A_qr_meas(2*n_dim+1) = sqrt(Rd);
176
177     [~, S_yy] = qr(A_qr_meas', 0);
178     S_yy = S_yy(1, 1);
179     S_yy = cholupdate(S_yy, sqrt(w_c(1)) * (Y_minus(1) - y_minus_srukf)
180 , '+'');
181
182     P_xy = zeros(n_dim, 1);
183     for i = 1:2*n_dim+1
184         P_xy = P_xy + w_c(i) * (Chi_minus_curr(:,i) - x_minus_srukf) *
185 (Y_minus(i) - y_minus_srukf)';
186     end
187
188     K_srukf = (P_xy / S_yy) / S_yy';
189
190     y_k = x_true(1, k+1) + v_mc(k+1);
191     x_hat_srukf(:, k+1) = x_minus_srukf + K_srukf * (y_k -
192 y_minus_srukf);
193
194     U_mat = K_srukf * S_yy';
195     S_srukf = S_minus_srukf;
196     for col = 1:size(U_mat, 2)
197         S_srukf = cholupdate(S_srukf, U_mat(:, col), '-');
198     end
199 else
200     x_hat_srukf(:, k+1) = x_minus_srukf;
201     S_srukf = S_minus_srukf;
202 end
203
204 err_srukf_mc = x_true - x_hat_srukf;
205 squared_errors_srukf = squared_errors_srukf + err_srukf_mc.^2;
206
207 if m == 1
208     x_true_rep = x_true;
209     x_hat_rep = x_hat;
210     P_hat_rep = P_hat;
211
212     y_meas_rep(steps_meas:steps_meas:N) = x_true(1, steps_meas:steps_meas:N
213 ) + v_mc(steps_meas:steps_meas:N);
214     x_hat_srukf_rep = x_hat_srukf;
215 end
216 end
217 elapsed_total = toc;
218
219 rmse_ekf = sqrt(squared_errors / M);
220 rmse_srukf = sqrt(squared_errors_srukf / M);
221
222 overall_rmse_ekf = sqrt(mean(rmse_ekf.^2, 2));
223 overall_rmse_srukf = sqrt(mean(rmse_srukf.^2, 2));
224

```

```

225 %saves
226 save_dir_31 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    InertialNavigation\HWs\HW6\SimulationFigures\3\3.1';
227 save_dir_32 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    InertialNavigation\HWs\HW6\SimulationFigures\3\3.2';
228 if ~exist(save_dir_31, 'dir'), mkdir(save_dir_31); end
229 if ~exist(save_dir_32, 'dir'), mkdir(save_dir_32); end
230
231 %plots
232 fig1 = figure;
233 subplot(2,1,1);
234 plot(t, x_true_rep(1,:), 'b', 'LineWidth', 1); hold on;
235 plot(t, x_hat_rep(1,:), 'r--', 'LineWidth', 1.2);
236 plot(t, y_meas_rep, 'g.', 'MarkerSize', 8);
237 xlabel('Time [s]'); ylabel('x_1');
238 legend('Ground Truth', 'A Posteriori Estimate', 'Measurement'); grid on;
239 title('Part 3.1: State x_1 - EKF Tracking & Self-Correction from Offset Initial
    Guess');
240
241 subplot(2,1,2);
242 plot(t, x_true_rep(2,:), 'b', 'LineWidth', 1); hold on;
243 plot(t, x_hat_rep(2,:), 'r--', 'LineWidth', 1.2);
244 xlabel('Time [s]'); ylabel('x_2');
245 legend('Ground Truth', 'A Posteriori Estimate'); grid on;
246 title('Part 3.1: State x_2 - EKF Tracking (Unmeasured State Convergence)');
247 saveas(fig1, fullfile(save_dir_31, 'ekf_state_tracking.png'));
248
249
250 err_rep = x_true_rep - x_hat_rep;
251 sigma_rep = zeros(2, N+1);
252 for k = 1:(N+1)
253     sigma_rep(1,k) = sqrt(P_hat_rep(1,1,k));
254     sigma_rep(2,k) = sqrt(P_hat_rep(2,2,k));
255 end
256
257 fig2 = figure;
258 subplot(2,1,1);
259 plot(t, err_rep(1,:), 'b'); hold on;
260 plot(t, 3*sigma_rep(1,:), 'r--', t, -3*sigma_rep(1,:), 'r--');
261 xlabel('Time [s]'); ylabel('x_1 Error');
262 legend('Estimation Error', '\pm 3\sigma Bound'); grid on;
263 title('Part 3.1: Estimation Error for x_1 with Analytical Converging Envelope')
    ;
264
265 subplot(2,1,2);
266 plot(t, err_rep(2,:), 'b'); hold on;
267 plot(t, 3*sigma_rep(2,:), 'r--', t, -3*sigma_rep(2,:), 'r--');
268 xlabel('Time [s]'); ylabel('x_2 Error');
269 legend('Estimation Error', '\pm 3\sigma Bound'); grid on;
270 title('Part 3.1: Estimation Error for x_2 with Analytical Converging Envelope')
    ;
271 saveas(fig2, fullfile(save_dir_31, 'ekf_error_bounds.png'));
272
273
274 fig3 = figure;
275 subplot(2,1,1);

```

```

276 plot(t, rmse_ekf(1,:), 'b', 'LineWidth', 1.5);
277 xlabel('Time [s]'); ylabel('RMSE of x_1'); grid on;
278 title(sprintf('Part 3.1: Monte Carlo RMSE for x_1 (EKF, M = %d trials)', M));
279
280 subplot(2,1,2);
281 plot(t, rmse_ekf(2,:), 'b', 'LineWidth', 1.5);
282 xlabel('Time [s]'); ylabel('RMSE of x_2'); grid on;
283 title(sprintf('Part 3.1: Monte Carlo RMSE for x_2 (EKF, M = %d trials)', M));
284 saveas(fig3, fullfile(save_dir_31, 'ekf_rmse_montecarlo.png'));
285
286
287 fig4 = figure;
288 subplot(2,1,1);
289 plot(t, rmse_ekf(1,:), 'r', 'LineWidth', 1); hold on;
290 plot(t, rmse_srufk(1,:), 'b--', 'LineWidth', 1.2);
291 xlabel('Time [s]'); ylabel('RMSE of x_1');
292 legend('EKF Performance', 'SRUKF Performance'); grid on;
293 title(sprintf('Part 3.2: RMSE Comparison for State x_1 (%d Monte Carlo Runs)',
294             M));
295
296 subplot(2,1,2);
297 plot(t, rmse_ekf(2,:), 'r', 'LineWidth', 1); hold on;
298 plot(t, rmse_srufk(2,:), 'b--', 'LineWidth', 1.2);
299 xlabel('Time [s]'); ylabel('RMSE of x_2');
300 legend('EKF Performance', 'SRUKF Performance'); grid on;
301 title(sprintf('Part 3.2: RMSE Comparison for State x_2 (%d Monte Carlo Runs)',
302             M));
303 saveas(fig4, fullfile(save_dir_32, 'ekf_vs_srufk_rmse.png'));
304
305
306 fig5 = figure;
307 subplot(2,1,1);
308 plot(t, x_true_rep(1,:), 'b', 'LineWidth', 1); hold on;
309 plot(t, x_hat_srufk_rep(1,:), 'm--', 'LineWidth', 1.2);
310 plot(t, y_meas_rep, 'g.', 'MarkerSize', 8);
311 xlabel('Time [s]'); ylabel('x_1');
312 legend('Ground Truth', 'SRUKF Estimate', 'Measurement'); grid on;
313 title('Part 3.2: State x_1 - SRUKF State Tracking & Convergence from Offset');
314
315 subplot(2,1,2);
316 plot(t, x_true_rep(2,:), 'b', 'LineWidth', 1); hold on;
317 plot(t, x_hat_srufk_rep(2,:), 'm--', 'LineWidth', 1.2);
318 xlabel('Time [s]'); ylabel('x_2');
319 legend('Ground Truth', 'SRUKF Estimate'); grid on;
320 title('Part 3.2: State x_2 - SRUKF Unmeasured State Tracking');
321 saveas(fig5, fullfile(save_dir_32, 'srufk_state_tracking.png'));
322
323 %prints
324 fprintf('\n part 3.1 summary: EKF Monte Carlo performance (M = %d trials) \n',
325         M);
326 fprintf('Overall Time-Averaged RMSE x1: %.4f\n', overall_rmse_ekf(1));
327 fprintf('Overall Time-Averaged RMSE x2: %.4f\n', overall_rmse_ekf(2));
328
329 fprintf('\n Part 3.2 Summary: SRUKF Monte carlo Performance (M = %d) \n', M);
330 fprintf('Overall Time-averaged RMSE x1 (SRUKF): %.4f (EKF was: %.4f)\n',
331         overall_rmse_srufk(1), overall_rmse_ekf(1));

```

```

328 fprintf('overall time-Averaged RMSE x2 (SRUKF): %.4f (EKF was: %.4f)\n',
    overall_rmse_srckf(2), overall_rmse_ekf(2));
329
330
331 %problem 3.3
332
333 % SRCKF parameter Allocation (Slides 89-92)
334 zeta = sqrt(n_dim) * [eye(n_dim), -eye(n_dim)]; % n_dim x 2*n_dim
335 %monte carlo
336 squared_errors_srckf = zeros(2, N+1); % Accumulator for SRCKF RMSE
337
338
339 x_hat_srckf_rep = zeros(2, N+1);
340
341 rng(1);
342
343
344 tic;
345 for m = 1:M
346
347     x_true = zeros(2, N+1);
348     x_true(:,1) = x0;
349     w_mc = Qd_sqrt * randn(2, N);
350     for k = 1:N
351         x_true(:,k+1) = x_true(:,k) + dt * f(x_true(:,k)) + w_mc(:,k);
352     end
353
354
355     v_mc = sqrt(Rd) * randn(1, N+1);
356
357     %srckf loop
358     x_hat_srckf = zeros(2, N+1);
359     x_hat_srckf(:,1) = x_hat0;
360     S_srckf = chol(P0, 'upper'); % initialization (slide 89)
361
362     for k = 1:N
363         % time update , slide 90
364         Chi_km1 = kron(x_hat_srckf(:,k), ones(1, 2*n_dim)) + S_srckf' * zeta;
365
366         Chi_star = zeros(n_dim, 2*n_dim);
367         for i = 1:2*n_dim
368             Chi_star(:,i) = Chi_km1(:,i) + dt * f(Chi_km1(:,i));
369         end
370
371         x_minus_srckf = sum(Chi_star, 2) / (2*n_dim);
372
373         A_qr_time = [ (1/sqrt(2*n_dim)) * (Chi_star - kron(x_minus_srckf, ones
374             (1,2*n_dim))), ...
375             sqrtm(Qd)' ];
376         [~, S_minus_srckf] = qr(A_qr_time', 0);
377         S_minus_srckf = S_minus_srckf(1:n_dim, 1:n_dim);
378
379         % measurement update , slide 91
380         is_meas_step = (mod(k, steps_meas) == 0);
381         if is_meas_step

```

```

381     Chi_minus = kron(x_minus_srckf, ones(1, 2*n_dim)) + S_minus_srckf'
    * zeta;
382
383     Z_minus = Chi_minus(1, :); %scalar measurement
384     z_hat_srckf = sum(Z_minus, 2) / (2*n_dim);
385
386     A_qr_meas = [ (1/sqrt(2*n_dim)) * (Z_minus - z_hat_srckf), sqrt(Rd)
    ];
387     [~, S_zz] = qr(A_qr_meas', 0);
388     S_zz = S_zz(1,1);
389
390     P_xz = zeros(n_dim, 1);
391     for i = 1:2*n_dim
392         P_xz = P_xz + (1/(2*n_dim)) * (Chi_minus(:,i) - x_minus_srckf)
    * (Z_minus(i) - z_hat_srckf)';
393     end
394
395     K_srckf = (P_xz / S_zz) / S_zz';
396
397     y_k = x_true(1, k+1) + v_mc(k+1);
398     x_hat_srckf(:, k+1) = x_minus_srckf + K_srckf * (y_k - z_hat_srckf)
    ;
399
400     A_qr_upd = [ (1/sqrt(2*n_dim)) * (Chi_minus - kron(x_minus_srckf,
    ones(1,2*n_dim))) ...
401                 - K_srckf * (1/sqrt(2*n_dim)) * (Z_minus - z_hat_srckf
    ), ...
402                 K_srckf * sqrt(Rd) ];
403     [~, S_srckf] = qr(A_qr_upd', 0);
404     S_srckf = S_srckf(1:n_dim, 1:n_dim);
405     else
406         x_hat_srckf(:, k+1) = x_minus_srckf;
407         S_srckf = S_minus_srckf;
408     end
409 end
410
411 err_srckf_mc = x_true - x_hat_srckf;
412 squared_errors_srckf = squared_errors_srckf + err_srckf_mc.^2;
413
414 if m == 1
415     x_hat_srckf_rep = x_hat_srckf;
416 end
417 end
418 elapsed_total_srckf = toc;
419
420
421
422 rmse_srckf = sqrt(squared_errors_srckf / M);
423 overall_rmse_srckf = sqrt(mean(rmse_srckf.^2, 2));
424
425 %save
426 save_dir_33 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    InertialNavigation\HWs\HW6\SimulationFigures\3\3.3';
427 if ~exist(save_dir_33, 'dir'), mkdir(save_dir_33); end
428
429 %plots

```

```

430 fig6 = figure;
431 subplot(2,1,1);
432 plot(t, x_true_rep(1,:), 'b', 'LineWidth', 1); hold on;
433 plot(t, x_hat_srckf_rep(1,:), 'k--', 'LineWidth', 1.2);
434 plot(t, y_meas_rep, 'g.', 'MarkerSize', 8);
435 xlabel('Time [s]'); ylabel('x_1');
436 legend('Ground Truth', 'SRCKF Estimate', 'Measurement'); grid on;
437 title('Part 3.3: State x_1 - SRCKF State Tracking & Convergence from Offset');
438
439 subplot(2,1,2);
440 plot(t, x_true_rep(2,:), 'b', 'LineWidth', 1); hold on;
441 plot(t, x_hat_srckf_rep(2,:), 'k--', 'LineWidth', 1.2);
442 xlabel('Time [s]'); ylabel('x_2');
443 legend('Ground Truth', 'SRCKF Estimate'); grid on;
444 title('Part 3.3: State x_2 - SRCKF Unmeasured State Tracking');
445 saveas(fig6, fullfile(save_dir_33, 'srckf_state_tracking.png'));
446
447
448 fig7 = figure;
449 subplot(2,1,1);
450 plot(t, rmse_ekf(1,:), 'r', 'LineWidth', 1); hold on;
451 plot(t, rmse_sruckf(1,:), 'b--', 'LineWidth', 1.2);
452 plot(t, rmse_srckf(1,:), 'k-.', 'LineWidth', 1.2);
453 xlabel('Time [s]'); ylabel('RMSE of x_1');
454 legend('EKF Performance', 'SRUKF Performance', 'SRCKF Performance'); grid on;
455 title(sprintf('Part 3.3: RMSE Comparison for State x_1 (%d Monte Carlo Runs)',
456 M));
457
458 subplot(2,1,2);
459 plot(t, rmse_ekf(2,:), 'r', 'LineWidth', 1); hold on;
460 plot(t, rmse_sruckf(2,:), 'b--', 'LineWidth', 1.2);
461 plot(t, rmse_srckf(2,:), 'k-.', 'LineWidth', 1.2);
462 xlabel('Time [s]'); ylabel('RMSE of x_2');
463 legend('EKF Performance', 'SRUKF Performance', 'SRCKF Performance'); grid on;
464 title(sprintf('Part 3.3: RMSE Comparison for State x_2 (%d Monte Carlo Runs)',
465 M));
466 saveas(fig7, fullfile(save_dir_33, 'ekf_vs_sruckf_vs_srckf_rmse.png'));
467
468 %prints
469 fprintf('\n Part 3.3 Summary: SRCKF Monte Carlo Performance (M = %d) \n', M);
470 fprintf('Overall Time-Averaged RMSE x1 (SRCKF): %.4f (EKF: %.4f | SRUKF: %.4f\n', ...
471 overall_rmse_srckf(1), overall_rmse_ekf(1), overall_rmse_sruckf(1));
472 fprintf('Overall Time-Averaged RMSE x2 (SRCKF): %.4f (EKF: %.4f | SRUKF: %.4f\n', ...
473 overall_rmse_srckf(2), overall_rmse_ekf(2), overall_rmse_sruckf(2));
474
475
476
477 %Q3.4
478 %as we had in slide 113: "A Monte Carlo approximation can be
479 % applied to implement the theoretical posterior CRLB... average over
480 % this ensemble" .
481

```

```

482 J_k = inv(P0); % (Slide 109)
483 PCRLB = zeros(2, 2, N+1);
484 PCRLB(:,:,1) = inv(J_k);
485
486 rng(1);
487
488
489 tic;
490
491 % Slide 112: "Jacobian of f_k(x_k) at the
492 % true state x_k".
493 % slide 112 remark: "the bound is
494 % independent of the actual measurement sequence"
495
496
497 x_true_all = zeros(2, N+1, M);
498 for m = 1:M
499     x_true_m = zeros(2, N+1);
500     x_true_m(:,1) = x0;
501     w_mc = sqrtm(Qd) * randn(2, N);
502     for k = 1:N
503         x_true_m(:,k+1) = x_true_m(:,k) + dt * f(x_true_m(:,k)) + w_mc(:,k);
504     end
505     x_true_all(:,:,m) = x_true_m;
506     sqrt_Rd_unused = sqrt(Rd) * randn(1, N+1);
507
508
509
510 end
511
512 Qd_inv = inv(Qd);
513 H_lin = [1, 0];
514
515 for k = 1:N
516     % slide 112
517     D11_acc = zeros(2,2);
518     D12_acc = zeros(2,2);
519     for m = 1:M
520         xk1 = x_true_all(1,k,m);
521         xk2 = x_true_all(2,k,m);
522         Fc_true = [0, 1;
523                   -1 - 2*xk1*xk2, 1 - xk1^2];
524         Ftilde = eye(2) + dt * Fc_true;
525
526         D11_acc = D11_acc + Ftilde' * Qd_inv * Ftilde;
527         D12_acc = D12_acc + (-Ftilde' * Qd_inv);
528     end
529     D11_k = D11_acc / M;
530     D12_k = D12_acc / M;
531     D21_k = D12_k';
532
533
534     is_meas_step = (mod(k, steps_meas) == 0);
535     if is_meas_step
536         D22_k = Qd_inv + H_lin' * (1/Rd) * H_lin;
537     else

```

```

538     D22_k = Qd_inv;
539     end
540
541     % recursive update
542     J_k = D22_k - D21_k * inv(J_k + D11_k) * D12_k;    % (Slide 107)
543     PCRLB(:, :, k+1) = inv(J_k);
544 end
545
546 elapsed_crlb = toc;
547 fprintf('PCRLB recursion run time (M = %d trials): %.3f s\n', M, elapsed_crlb);
548
549 % slide 105
550 crlb_std = zeros(2, N+1);
551 for k = 1:(N+1)
552     crlb_std(1,k) = sqrt(PCRLB(1,1,k));
553     crlb_std(2,k) = sqrt(PCRLB(2,2,k));
554 end
555
556 %save
557 save_dir_34 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
    InertialNavigation\HWs\HW6\SimulationFigures\3\3.4';
558 if ~exist(save_dir_34, 'dir'), mkdir(save_dir_34); end
559
560 %plots
561 fig8 = figure;
562 subplot(2,1,1);
563 plot(t, rmse_ekf(1,:), 'r', 'LineWidth', 1);    hold on;
564 plot(t, rmse_srufk(1,:), 'b--', 'LineWidth', 1);
565 plot(t, rmse_srckf(1,:), 'm:', 'LineWidth', 1.4);
566 plot(t, crlb_std(1,:), 'k-.', 'LineWidth', 1.6);
567 xlabel('Time [s]'); ylabel('RMSE / CRLB of x_1');
568 legend('EKF RMSE', 'SRUFK RMSE', 'SRCKF RMSE', 'Theoretical CRLB (1\sigma)');
    grid on;
569 title(sprintf('Part 3.4: RMSE vs. Posterior CRLB for State x_1 (%d Monte Carlo
    Runs)', M));
570
571 subplot(2,1,2);
572 plot(t, rmse_ekf(2,:), 'r', 'LineWidth', 1);    hold on;
573 plot(t, rmse_srufk(2,:), 'b--', 'LineWidth', 1);
574 plot(t, rmse_srckf(2,:), 'm:', 'LineWidth', 1.4);
575 plot(t, crlb_std(2,:), 'k-.', 'LineWidth', 1.6);
576 xlabel('Time [s]'); ylabel('RMSE / CRLB of x_2');
577 legend('EKF RMSE', 'SRUFK RMSE', 'SRCKF RMSE', 'Theoretical CRLB (1\sigma)');
    grid on;
578 title(sprintf('Part 3.4: RMSE vs. Posterior CRLB for State x_2 (%d Monte Carlo
    Runs)', M));
579 saveas(fig8, fullfile(save_dir_34, 'rmse_vs_crlb_full.png'));
580
581
582 t_zoom_start = 20;    % skip the initial-condition-recovery transient
583 t_zoom_end   = 30;    % a representative 10 s steady-state window
584 idx_zoom_34  = (t >= t_zoom_start) & (t <= t_zoom_end);
585
586 fig9 = figure;
587 subplot(2,1,1);

```

```

588 plot(t(idx_zoom_34), rmse_ekf(1,idx_zoom_34), 'r-o', 'LineWidth', 1, '
    MarkerSize', 3); hold on;
589 plot(t(idx_zoom_34), rmse_srufk(1,idx_zoom_34), 'b--s', 'LineWidth', 1, '
    MarkerSize', 3);
590 plot(t(idx_zoom_34), rmse_srckf(1,idx_zoom_34), 'm:^', 'LineWidth', 1.2, '
    MarkerSize', 3);
591 plot(t(idx_zoom_34), crlb_std(1,idx_zoom_34), 'k-.', 'LineWidth', 1.8);
592 xlabel('Time [s]'); ylabel('RMSE / CRLB of x_1');
593 legend('EKF RMSE', 'SRUFK RMSE', 'SRCKF RMSE', 'Theoretical CRLB (1\sigma)');
    grid on;
594 title(sprintf('Part 3.4: Zoomed Steady-State Comparison for x_1 (t = %d-%d s)',
    t_zoom_start, t_zoom_end));
595
596 subplot(2,1,2);
597 plot(t(idx_zoom_34), rmse_ekf(2,idx_zoom_34), 'r-o', 'LineWidth', 1, '
    MarkerSize', 3); hold on;
598 plot(t(idx_zoom_34), rmse_srufk(2,idx_zoom_34), 'b--s', 'LineWidth', 1, '
    MarkerSize', 3);
599 plot(t(idx_zoom_34), rmse_srckf(2,idx_zoom_34), 'm:^', 'LineWidth', 1.2, '
    MarkerSize', 3);
600 plot(t(idx_zoom_34), crlb_std(2,idx_zoom_34), 'k-.', 'LineWidth', 1.8);
601 xlabel('Time [s]'); ylabel('RMSE / CRLB of x_2');
602 legend('EKF RMSE', 'SRUFK RMSE', 'SRCKF RMSE', 'Theoretical CRLB (1\sigma)');
    grid on;
603 title(sprintf('Part 3.4: Zoomed Steady-State Comparison for x_2 (t = %d-%d s)',
    t_zoom_start, t_zoom_end));
604 saveas(fig9, fullfile(save_dir_34, 'rmse_vs_crlb_zoomed.png'));
605
606 %print
607 overall_crlb = sqrt(mean(crlb_std.^2, 2));
608 fprintf('\n Part 3.4 Summary: RMSE vs. Posterior CRLB \n');
609 fprintf('Overall Theoretical CRLB (1-sigma) x1: %.4f | x2: %.4f\n',
    overall_crlb(1), overall_crlb(2));
610 fprintf('Overall RMSE x1 -- EKF: %.4f | SRUFK: %.4f | SRCKF: %.4f\n', ...
    overall_rmse_ekf(1), overall_rmse_srufk(1), overall_rmse_srckf(1));
612 fprintf('Overall RMSE x2 -- EKF: %.4f | SRUFK: %.4f | SRCKF: %.4f\n', ...
    overall_rmse_ekf(2), overall_rmse_srufk(2), overall_rmse_srckf(2));
613
614
615
616
617 %3.5
618 rng(7);
619 x_true_bench = zeros(2, N+1);
620 x_true_bench(:,1) = x0;
621 w_bench = sqrtm(Qd) * randn(2, N);
622 for k = 1:N
623     x_true_bench(:,k+1) = x_true_bench(:,k) + dt * f(x_true_bench(:,k)) +
        w_bench(:,k);
624 end
625 v_bench = sqrt(Rd) * randn(1, N+1);
626
627 run_ekf_once = @() run_single_ekf(f, dt, N, x_hat0, P0, Qd, Rd, steps_meas,
    x_true_bench, v_bench);
628 run_srufk_once = @() run_single_srufk(f, dt, N, n_dim, kappa, w_m, w_c, x_hat0,
    P0, Qd, Rd, steps_meas, x_true_bench, v_bench);

```

```

629 run_srckf_once = @() run_single_srckf(f, dt, N, n_dim, zeta, x_hat0, P0, Qd, Rd
    , steps_meas, x_true_bench, v_bench);
630
631 %tic/ toc
632 fprintf('\n Part 3.5: Timing a single filter pass (tic/toc) \n');
633
634 tic; run_ekf_once(); t_single_ekf_tictoc = toc;
635 tic; run_srufk_once(); t_single_srufk_tictoc = toc;
636 tic; run_srckf_once(); t_single_srckf_tictoc = toc;
637
638 fprintf('Single-run tic/toc -- EKF: %.5f s | SRUFK: %.5f s | SRCKF: %.5f s\n',
    ...
639 t_single_ekf_tictoc, t_single_srufk_tictoc, t_single_srckf_tictoc);
640
641
642 fprintf('\n Part 3.5: Timing a single filter pass (timeit) \n');
643
644 t_single_ekf_timeit = timeit(run_ekf_once);
645 t_single_srufk_timeit = timeit(run_srufk_once);
646 t_single_srckf_timeit = timeit(run_srckf_once);
647
648 fprintf('Single-run timeit -- EKF: %.5f s | SRUFK: %.5f s | SRCKF: %.5f s\n',
    ...
649 t_single_ekf_timeit, t_single_srufk_timeit, t_single_srckf_timeit);
650
651
652 fprintf('\n Part 3.5: Timing the full M-trial Monte Carlo loop (tic/toc) \n');
653
654 rng(1);
655 tic;
656 for m = 1:M
657     x_true_mc = zeros(2, N+1);
658     x_true_mc(:,1) = x0;
659     w_mc = sqrtm(Qd) * randn(2, N);
660     for k = 1:N
661         x_true_mc(:,k+1) = x_true_mc(:,k) + dt * f(x_true_mc(:,k)) + w_mc(:,k);
662     end
663     v_mc = sqrt(Rd) * randn(1, N+1);
664     run_single_ekf(f, dt, N, x_hat0, P0, Qd, Rd, steps_meas, x_true_mc, v_mc);
665 end
666 t_mc_ekf_tictoc = toc;
667
668 rng(1);
669 tic;
670 for m = 1:M
671     x_true_mc = zeros(2, N+1);
672     x_true_mc(:,1) = x0;
673     w_mc = sqrtm(Qd) * randn(2, N);
674     for k = 1:N
675         x_true_mc(:,k+1) = x_true_mc(:,k) + dt * f(x_true_mc(:,k)) + w_mc(:,k);
676     end
677     v_mc = sqrt(Rd) * randn(1, N+1);
678     run_single_srufk(f, dt, N, n_dim, kappa, w_m, w_c, x_hat0, P0, Qd, Rd,
        steps_meas, x_true_mc, v_mc);
679 end
680 t_mc_srufk_tictoc = toc;

```

```

681
682 rng(1);
683 tic;
684 for m = 1:M
685     x_true_mc      = zeros(2, N+1);
686     x_true_mc(:,1) = x0;
687     w_mc           = sqrtm(Qd) * randn(2, N);
688     for k = 1:N
689         x_true_mc(:,k+1) = x_true_mc(:,k) + dt * f(x_true_mc(:,k)) + w_mc(:,k);
690     end
691     v_mc = sqrt(Rd) * randn(1, N+1);
692     run_single_srckf(f, dt, N, n_dim, zeta, x_hat0, P0, Qd, Rd, steps_meas,
        x_true_mc, v_mc);
693 end
694 t_mc_srckf_tictoc = toc;
695
696 fprintf('Full M=%d-trial MC  -- EKF: %.3f s | SRUKF: %.3f s | SRCKF: %.3f s\n',
        ...
697     M, t_mc_ekf_tictoc, t_mc_sruckf_tictoc, t_mc_srckf_tictoc);
698
699 %
700 save_dir_35 = 'C:\Users\Asus\OneDrive\Desktop\EE_Archive\6th_semester\
        InertialNavigation\HWs\HW6\SimulationFigures\3\3.5';
701 if ~exist(save_dir_35, 'dir'), mkdir(save_dir_35); end
702
703 fprintf('\n--- Part 3.5: Profiling each filter (MATLAB Profiler) ---\n');
704
705 profile('on');
706 for m = 1:10
707     run_ekf_once();
708 end
709 profile('off');
710 prof_ekf = profile('info');
711 profsave(prof_ekf, fullfile(save_dir_35, 'profiler_report_EKF'));
712
713 profile('on');
714 for m = 1:10
715     run_sruckf_once();
716 end
717 profile('off');
718 prof_sruckf = profile('info');
719 profsave(prof_sruckf, fullfile(save_dir_35, 'profiler_report_SRUKF'));
720
721 profile('on');
722 for m = 1:10
723     run_srckf_once();
724 end
725 profile('off');
726 prof_srckf = profile('info');
727 profsave(prof_srckf, fullfile(save_dir_35, 'profiler_report_SRCKF'));
728
729
730 t_prof_ekf_per_call = get_profiler_self_time(prof_ekf, 'run_single_ekf')
        / 10;
731 t_prof_sruckf_per_call = get_profiler_self_time(prof_sruckf, 'run_single_sruckf')
        / 10;

```

```

732 t_prof_srckf_per_call = get_profiler_self_time(prof_srckf, 'run_single_srckf')
    / 10;
733
734 fprintf('Profiler (per call) -- EKF: %.5f s | SRUKF: %.5f s | SRCKF: %.5f s\n',
    ...
735     t_prof_ekf_per_call, t_prof_sruckf_per_call, t_prof_srckf_per_call);
736 fprintf('Full Profiler HTML reports saved to: %s\n', save_dir_35);
737
738 %plots
739 filter_names = {'EKF', 'SRUKF', 'SRCKF'};
740 single_run_matrix = [t_single_ekf_tictoc,    t_single_ekf_timeit,
    t_prof_ekf_per_call; ...
741                     t_single_sruckf_tictoc, t_single_sruckf_timeit,
    t_prof_sruckf_per_call; ...
742                     t_single_srckf_tictoc, t_single_srckf_timeit,
    t_prof_srckf_per_call] * 1000; % ms
743
744 fig10 = figure;
745 b10 = bar(single_run_matrix, 'grouped');
746 b10(1).FaceColor = [0.85 0.33 0.10]; % tic/toc    -- orange
747 b10(2).FaceColor = [0.00 0.45 0.74]; % timeit     -- blue
748 b10(3).FaceColor = [0.47 0.67 0.19]; % profiler  -- green
749 set(gca, 'XTickLabel', filter_names);
750 ylabel('Time per single filter run [ms]');
751 legend('tic/toc', 'timeit', 'Profiler (self-time)', 'Location', 'northwest');
752 grid on;
753 title('Part 3.5: Single-Run Computational Cost -- Cross-Tool Comparison');
754 for i = 1:3
755     for j = 1:3
756         text(i + (j-2)*0.225, single_run_matrix(i,j) + 0.02*max(
    single_run_matrix(:)), ...
757             sprintf('%.3f', single_run_matrix(i,j)), 'HorizontalAlignment', '
    center', 'FontSize', 8);
758     end
759 end
760 saveas(fig10, fullfile(save_dir_35, 'single_run_cost_comparison.png'));
761
762
763 mc_times = [t_mc_ekf_tictoc, t_mc_sruckf_tictoc, t_mc_srckf_tictoc];
764
765 fig11 = figure;
766 b11 = bar(mc_times, 'FaceColor', 'flat');
767 b11.CData = [0.85 0.33 0.10; 0.00 0.45 0.74; 0.47 0.67 0.19];
768 set(gca, 'XTickLabel', filter_names);
769 ylabel(sprintf('Total time for M = %d trials [s]', M));
770 grid on;
771 title(sprintf('Part 3.5: Full Monte Carlo Computational Cost (tic/toc, M = %d
    trials)', M));
772 for i = 1:3
773     text(i, mc_times(i) + 0.02*max(mc_times), sprintf('%.3f s', mc_times(i)),
    ...
774         'HorizontalAlignment', 'center', 'FontSize', 9, 'FontWeight', 'bold');
775 end
776 saveas(fig11, fullfile(save_dir_35, 'monte_carlo_cost_comparison.png'));
777
778

```

```

779 relative_cost_single = single_run_matrix(:,2) / single_run_matrix(1,2);
780 relative_cost_mc      = mc_times / mc_times(1);
781
782 fig12 = figure;
783 subplot(1,2,1);
784 b12a = bar(relative_cost_single, 'FaceColor', 'flat');
785 b12a.CData = [0.85 0.33 0.10; 0.00 0.45 0.74; 0.47 0.67 0.19];
786 set(gca, 'XTickLabel', filter_names);
787 ylabel('Relative cost (EKF = 1\times)');
788 title('Single-Run Relative Cost'); grid on;
789 yline(1, 'k--', 'LineWidth', 1);
790 for i = 1:3
791     text(i, relative_cost_single(i) + 0.05, sprintf('%.2f\times',
792         relative_cost_single(i)), ...
793         'HorizontalAlignment', 'center', 'FontSize', 9);
794 end
795 subplot(1,2,2);
796 b12b = bar(relative_cost_mc, 'FaceColor', 'flat');
797 b12b.CData = [0.85 0.33 0.10; 0.00 0.45 0.74; 0.47 0.67 0.19];
798 set(gca, 'XTickLabel', filter_names);
799 ylabel('Relative cost (EKF = 1\times)');
800 title('Full Monte Carlo Relative Cost'); grid on;
801 yline(1, 'k--', 'LineWidth', 1);
802 for i = 1:3
803     text(i, relative_cost_mc(i) + 0.05, sprintf('%.2f\times', relative_cost_mc
804         (i)), ...
805         'HorizontalAlignment', 'center', 'FontSize', 9);
806 end
807 sgtitle('Part 3.5: Relative Computational Overhead Normalized to EKF');
808 saveas(fig12, fullfile(save_dir_35, 'relative_cost_comparison.png'));
809 %prints
810 fprintf(' Part 3.5 Summary: Computational Cost Comparison (EKF/SRUKF/SRCKF)\n')
811 ;
812 fprintf('%-10s | %12s | %12s | %14s | %14s\n', 'Filter', 'tic/toc[ms]', 'timeit
813     [ms]', 'Profiler[ms]', sprintf('MC (M=%d)[s]', M));
814 fprintf('-----\n');
815 fprintf('%-10s | %12.4f | %12.4f | %14.4f | %14.3f\n', 'EKF',
816     t_single_ekf_tictoc*1000, t_single_ekf_timeit*1000, t_prof_ekf_per_call
817     *1000, t_mc_ekf_tictoc);
818 fprintf('%-10s | %12.4f | %12.4f | %14.4f | %14.3f\n', 'SRUKF',
819     t_single_srufk_tictoc*1000, t_single_srufk_timeit*1000,
820     t_prof_srufk_per_call*1000, t_mc_srufk_tictoc);
821 fprintf('%-10s | %12.4f | %12.4f | %14.4f | %14.3f\n', 'SRCKF',
822     t_single_srckf_tictoc*1000, t_single_srckf_timeit*1000,
823     t_prof_srckf_per_call*1000, t_mc_srckf_tictoc);
824
825 fprintf('Figures saved to: %s\n', save_dir_35);
826 disp('Inspect single_run_cost_comparison.png (cross-tool agreement check),');
827 disp('monte_carlo_cost_comparison.png (aggregate M-trial cost), and');
828 disp('relative_cost_comparison.png (EKF-normalized overhead) for the');
829 disp('computational cost discussion required by Part 3.5.');
```

```

825 function self_time = get_profiler_self_time(p, fname)
826     all_names = {p.FunctionTable.FunctionName};
827
828     short_names = cellfun(@(s) regexp(s, '^.*>', ''), all_names, '
UniformOutput', false);
829     idx = find(strcmp(short_names, fname));
830     if isempty(idx)
831         warning('Profiler: function "%s" not found in FunctionTable; returning
NaN.', fname);
832         self_time = NaN;
833     else
834         self_time = p.FunctionTable(idx(1)).TotalTime;
835     end
836 end
837
838 function x_hat = run_single_ekf(f, dt, N, x_hat0, P0, Qd, Rd, steps_meas,
x_true, v_meas)
839     x_hat = zeros(2, N+1);
840     P_hat = zeros(2, 2, N+1);
841     x_hat(:,1) = x_hat0;
842     P_hat(:,:,1) = P0;
843     H_k = [1, 0];
844
845     for k = 1:N
846         x_prev = x_hat(:,k);
847         x1_prev = x_prev(1);
848         x2_prev = x_prev(2);
849
850         Fc_km1 = [0, 1;
851                 -1 - 2*x1_prev*x2_prev, 1 - x1_prev^2];
852         F_km1 = eye(2) + dt * Fc_km1;
853
854         x_minus = x_prev + dt * f(x_prev);
855         P_minus = F_km1 * P_hat(:,:,k) * F_km1' + Qd;
856
857         if mod(k, steps_meas) == 0
858             y_k = x_true(1, k+1) + v_meas(k+1);
859             S_k = H_k * P_minus * H_k' + Rd;
860             K_k = P_minus * H_k' / S_k;
861             nu = y_k - H_k * x_minus;
862             x_hat(:,k+1) = x_minus + K_k * nu;
863             P_hat(:,:,k+1) = (eye(2) - K_k * H_k) * P_minus;
864         else
865             x_hat(:,k+1) = x_minus;
866             P_hat(:,:,k+1) = P_minus;
867         end
868     end
869 end
870
871 function x_hat_srukf = run_single_srukf(f, dt, N, n_dim, kappa, w_m, w_c,
x_hat0, P0, Qd, Rd, steps_meas, x_true, v_meas)
872     x_hat_srukf = zeros(n_dim, N+1);
873     x_hat_srukf(:,1) = x_hat0;
874     S_srukf = chol(P0, 'upper');
875     gamma_scale = sqrt(n_dim + kappa);
876

```

```

877     for k = 1:N
878         Chi_prev = zeros(n_dim, 2*n_dim+1);
879         Chi_prev(:,1) = x_hat_srukf(:,k);
880         for i = 1:n_dim
881             Chi_prev(:, 1+i) = x_hat_srukf(:,k) + gamma_scale * S_srukf(i
882 ,:);
883             Chi_prev(:, 1+n_dim+i) = x_hat_srukf(:,k) - gamma_scale * S_srukf(i
884 ,:);
885         end
886
887         Chi_minus = zeros(n_dim, 2*n_dim+1);
888         for i = 1:2*n_dim+1
889             Chi_minus(:,i) = Chi_prev(:,i) + dt * f(Chi_prev(:,i));
890         end
891
892         x_minus_srukf = zeros(n_dim, 1);
893         for i = 1:2*n_dim+1
894             x_minus_srukf = x_minus_srukf + w_m(i) * Chi_minus(:,i);
895         end
896
897         A_qr_time = zeros(n_dim, 2*n_dim + n_dim);
898         for i = 1:2*n_dim
899             A_qr_time(:, i) = sqrt(w_c(1+i)) * (Chi_minus(:, 1+i) -
900 x_minus_srukf);
901         end
902         A_qr_time(:, 2*n_dim+1:end) = sqrtm(Qd)';
903
904         [~, S_minus_srukf] = qr(A_qr_time', 0);
905         S_minus_srukf = S_minus_srukf(1:n_dim, 1:n_dim);
906         S_minus_srukf = cholupdate(S_minus_srukf, sqrt(w_c(1)) * (Chi_minus
907 (:,1) - x_minus_srukf), '+');
908
909         if mod(k, steps_meas) == 0
910             Chi_minus_curr = zeros(n_dim, 2*n_dim+1);
911             Chi_minus_curr(:,1) = x_minus_srukf;
912             for i = 1:n_dim
913                 Chi_minus_curr(:, 1+i) = x_minus_srukf + gamma_scale *
914 S_minus_srukf(i,:);
915                 Chi_minus_curr(:, 1+n_dim+i) = x_minus_srukf - gamma_scale *
916 S_minus_srukf(i,:);
917             end
918
919             Y_minus = zeros(1, 2*n_dim+1);
920             for i = 1:2*n_dim+1
921                 Y_minus(i) = Chi_minus_curr(1, i);
922             end
923
924             y_minus_srukf = 0;
925             for i = 1:2*n_dim+1
926                 y_minus_srukf = y_minus_srukf + w_m(i) * Y_minus(i);
927             end
928
929             A_qr_meas = zeros(1, 2*n_dim + 1);
930             for i = 1:2*n_dim
931                 A_qr_meas(i) = sqrt(w_c(1+i)) * (Y_minus(1+i) - y_minus_srukf);
932             end
933         end
934     end

```

```

927         A_qr_meas(2*n_dim+1) = sqrt(Rd);
928
929         [~, S_yy] = qr(A_qr_meas', 0);
930         S_yy = S_yy(1, 1);
931         S_yy = cholupdate(S_yy, sqrt(w_c(1)) * (Y_minus(1) - y_minus_sruckf)
, '+'');
932
933         P_xy = zeros(n_dim, 1);
934         for i = 1:2*n_dim+1
935             P_xy = P_xy + w_c(i) * (Chi_minus_curr(:,i) - x_minus_sruckf) *
(Y_minus(i) - y_minus_sruckf)';
936         end
937
938         K_sruckf = (P_xy / S_yy) / S_yy';
939         y_k = x_true(1, k+1) + v_meas(k+1);
940         x_hat_sruckf(:, k+1) = x_minus_sruckf + K_sruckf * (y_k -
y_minus_sruckf);
941
942         U_mat = K_sruckf * S_yy';
943         S_sruckf = S_minus_sruckf;
944         for col = 1:size(U_mat, 2)
945             S_sruckf = cholupdate(S_sruckf, U_mat(:, col), '-');
946         end
947     else
948         x_hat_sruckf(:, k+1) = x_minus_sruckf;
949         S_sruckf = S_minus_sruckf;
950     end
951 end
952 end
953
954 function x_hat_srckf = run_single_srckf(f, dt, N, n_dim, zeta, x_hat0, P0, Qd,
Rd, steps_meas, x_true, v_meas)
955     x_hat_srckf = zeros(n_dim, N+1);
956     x_hat_srckf(:,1) = x_hat0;
957     S_srckf = chol(P0, 'upper');
958
959     for k = 1:N
960         Chi_km1 = kron(x_hat_srckf(:,k), ones(1, 2*n_dim)) + S_srckf' * zeta;
961
962         Chi_star = zeros(n_dim, 2*n_dim);
963         for i = 1:2*n_dim
964             Chi_star(:,i) = Chi_km1(:,i) + dt * f(Chi_km1(:,i));
965         end
966
967         x_minus_srckf = sum(Chi_star, 2) / (2*n_dim);
968
969         A_qr_time = [ (1/sqrt(2*n_dim)) * (Chi_star - kron(x_minus_srckf, ones
(1,2*n_dim)))) , sqrtm(Qd)' ];
970         [~, S_minus_srckf] = qr(A_qr_time', 0);
971         S_minus_srckf = S_minus_srckf(1:n_dim, 1:n_dim);
972
973         if mod(k, steps_meas) == 0
974             Chi_minus = kron(x_minus_srckf, ones(1, 2*n_dim)) + S_minus_srckf'
* zeta;
975             Z_minus = Chi_minus(1, :);
976             z_hat_srckf = sum(Z_minus, 2) / (2*n_dim);

```

```

977         A_qr_meas = [ (1/sqrt(2*n_dim)) * (Z_minus - z_hat_srckf), sqrt(Rd)
978 ];
979         [~, S_zz] = qr(A_qr_meas', 0);
980         S_zz = S_zz(1,1);
981
982         P_xz = zeros(n_dim, 1);
983         for i = 1:2*n_dim
984             P_xz = P_xz + (1/(2*n_dim)) * (Chi_minus(:,i) - x_minus_srckf)
985             * (Z_minus(i) - z_hat_srckf)';
986         end
987
988         K_srckf = (P_xz / S_zz) / S_zz';
989         y_k = x_true(1, k+1) + v_meas(k+1);
990         x_hat_srckf(:, k+1) = x_minus_srckf + K_srckf * (y_k - z_hat_srckf)
991 ;
992
993         A_qr_upd = [ (1/sqrt(2*n_dim)) * (Chi_minus - kron(x_minus_srckf,
994 ones(1,2*n_dim))) ...
995                     - K_srckf * (1/sqrt(2*n_dim)) * (Z_minus - z_hat_srckf
996 ), K_srckf * sqrt(Rd) ];
997         [~, S_srckf] = qr(A_qr_upd', 0);
998         S_srckf = S_srckf(1:n_dim, 1:n_dim);
999     else
1000         x_hat_srckf(:, k+1) = x_minus_srckf;
1001     end
1002 end
1003 end

```

Listing 10: complete matlab code for all parts of problem 3

```

1  Part 3.5 Summary: Computational Cost Comparison (EKF/SRUKF/SRCKF)
2
3  Filter      |  tic/toc[ms] |  timeit[ms] |  Profiler[ms] |  MC (M=130)[s]
4  -----
5  EKF         |    48.0906 |    18.8949 |    30.2355 |    1.551
6  SRUKF       |   138.7373 |    62.7916 |   356.6895 |    8.866
7  SRCKF       |   131.3872 |    73.4287 |   360.7943 |   10.112
8  =====

```

Listing 11: print output of 3.5